第八章 实现行业问答智能问答技术方案

内容提要

那位科技

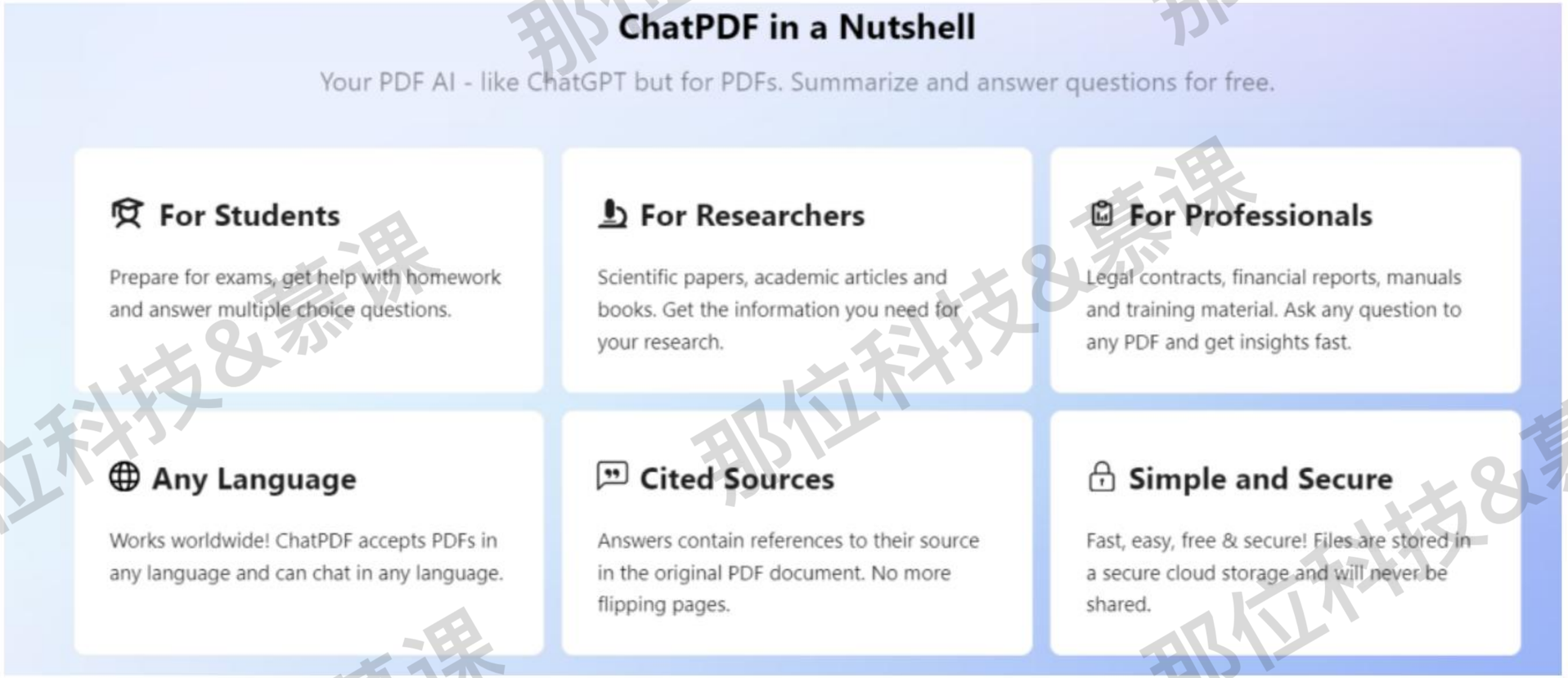
- 1. 大模型行业文档的几个应用场景
- 2. 大模型文档问答的常规实现技术方案
- 3. 大模型文档问答的几个小案例
- 5.大模型文档问答的几个优化策略

行业文档问答的几个应用场景

大模型之家

➢ChatPDF可以让用户更轻松地阅读文章，上传pdf文件后可以用类似于ChatGPT对话的方式获取其中信息

图表 16：ChatPDF 功能



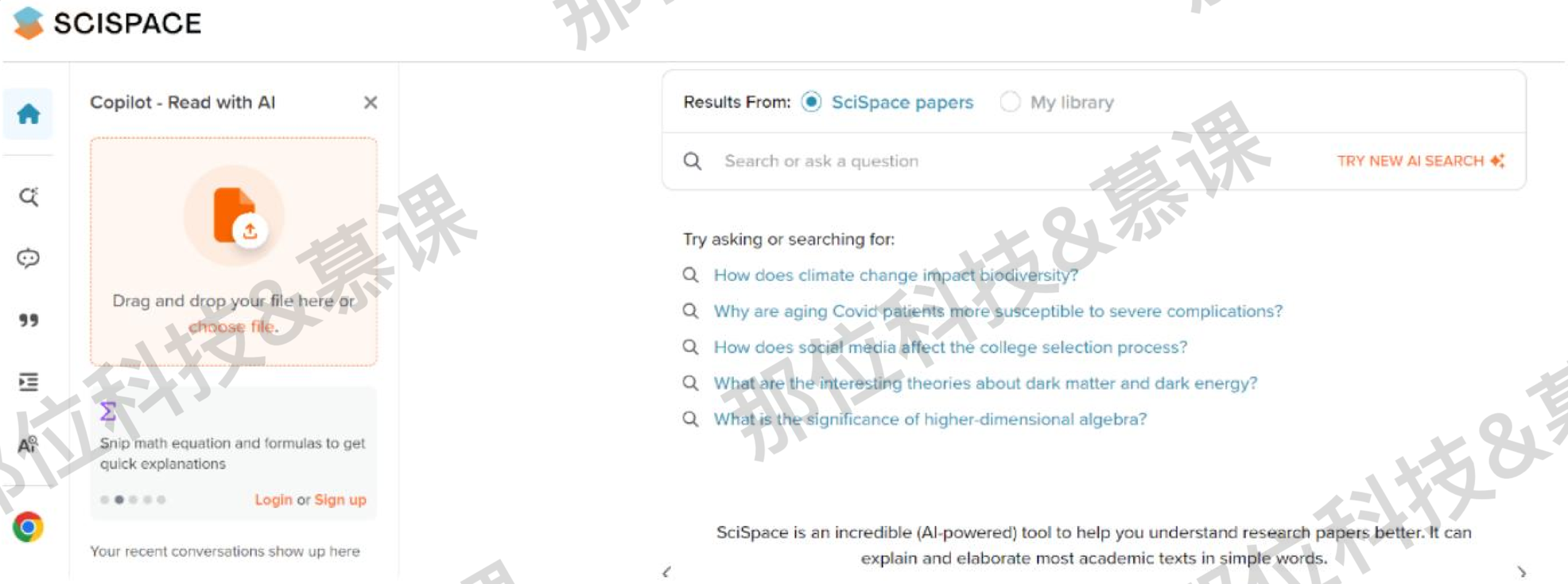
资料来源：ChatPDF 官网，国盛证券研究所

那位科技&慕课

大模型用于文档问答

大模型之家

➢SCISPACE可以利用人工智能帮助用户更好地理解研究论文，它可以用简单的语言解释 和阐述大多数学术论文，可以上传文件或是搜索文库。



资料来源：SCISPACE 官网，国盛证券研究所

那位科技&慕课

大模型之家

➢针对文档表格信息进行问答

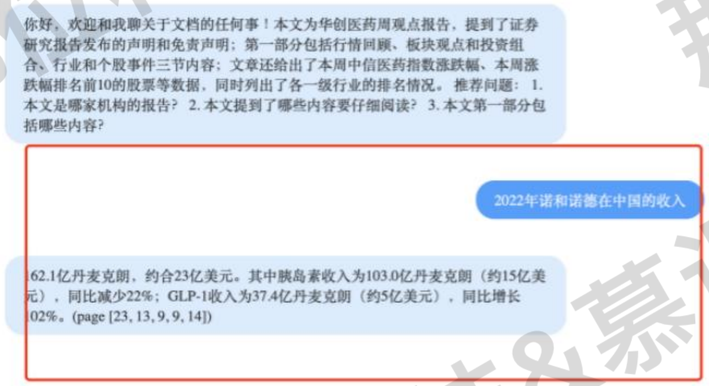


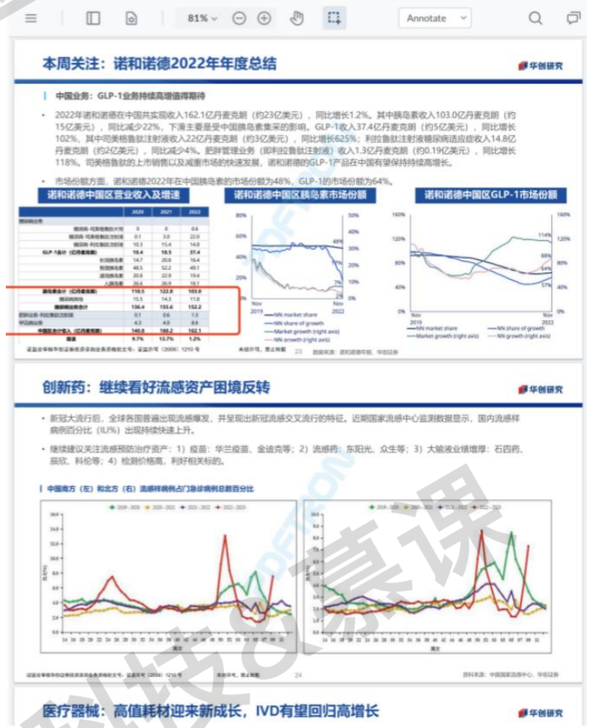


行权期	考核年度	定比 2021 年，2022-2024 年年度营业收入增长率或净利润增长率	
		目标增长率（Am）	触发增长率（An）
第一个行权期	2022	30.00%	22.50%
第二个行权期	2023	69.00%	51.75%
第三个行权期	2024	119.00%	89.25%
指标		指标完成度	行权系数（X）
营业收入增长率或净利润增长率 A		A≥Am	100%
		An≤A<Am	80%
		A<An	0%
各批次实际可行权额度		各批次计划行权额度×行权系数（X）	




```
'{"msg":"作为AI语言模型，我没有与复旦大学的直接关系或联系。我可以为任何需要语言处理和智能问答服务的个人或组织提供帮助。(page [1])","page":[1],"reftext":"\\begin{tabular}{lrll}\n\\toprule\n 行权期 & 考核年度 & 定比2021年，2022-2024年年度营业收入增长率或净利润增长率 & Unnamed: 3 \\\\\n\\midrule\n NaN & NaN & 目标增长率 (Am) & 触发增长率 (An) \\\\\n 第一个行权期 & 2022.0 & 30.00\\% & 22.50\\% \\\\\n 第二个行权期 & 2023.0 & 69.00\\% & 51.75\\% \\\\\n 第三个行权期 & 2024.0 & 119.00\\% & 89.25\\% \\\\\n 指标 & NaN & 指标完成度 & 行权系数（X） \\\\\n营业收入增长率或净利润 增长率A & NaN & A≥Am & 100\\% \\\\\n NaN & NaN & An≤A\\textbackslash n\\textbackslash n80\\% & NaN \\\\\n NaN & NaN & A\\textbackslash n\\textbackslash n0\\% & NaN \\\\\n NaN & NaN & 各批次计划行权额度×行权系数（X） & NaN \\\\\n 各批次实际可行权额度 5610 & NaN & NaN & NaN \\\\\n\\bottomrule\n\\end{tabular}"}'
```



大模型问答的常规实现方案

外挂文档问答的必要性

大模型之家

➢**微调Fine-Tuning**

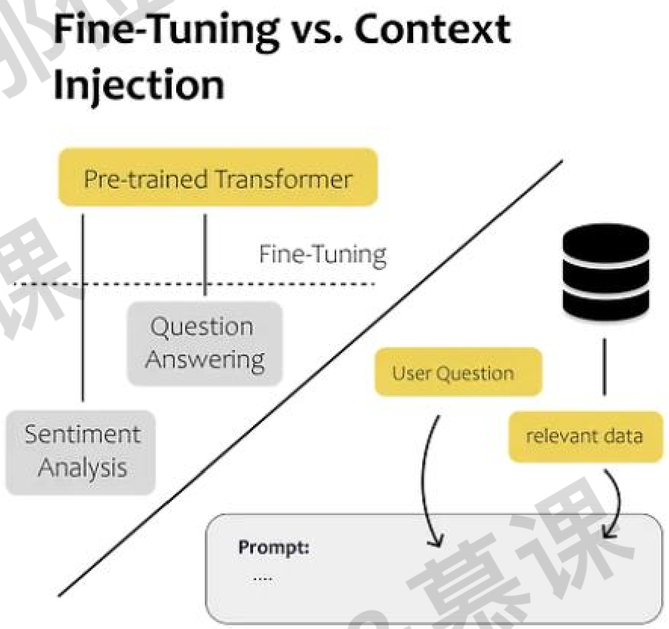
➢指使用其他数据训练现有语言模型，以针对特定任务对其进行优化。使用BERT或LLama等预训练模型，然后通过添加特定于用例的训练数据来适应特定任务的需求，而不是从头开始训练语言模型。

➢由于模型已经对大量的通用语言数据进行了训练，而特定域数据通常不足以覆盖模型已经学到的内容，这就是我们所说的学不进去的情况。

➢**当上下文注入**

➢即不修改LLM，专注于提示本身，并将相关上下文注入到提示中，让模型参考这个提示进行作答，但是其问题在于如何为提示提供正确的信息。目前我们所能看到的就是相关性召回，其有个假设，即问题的答案在召回的最相似的文档里。

➢|



那位科技&慕课

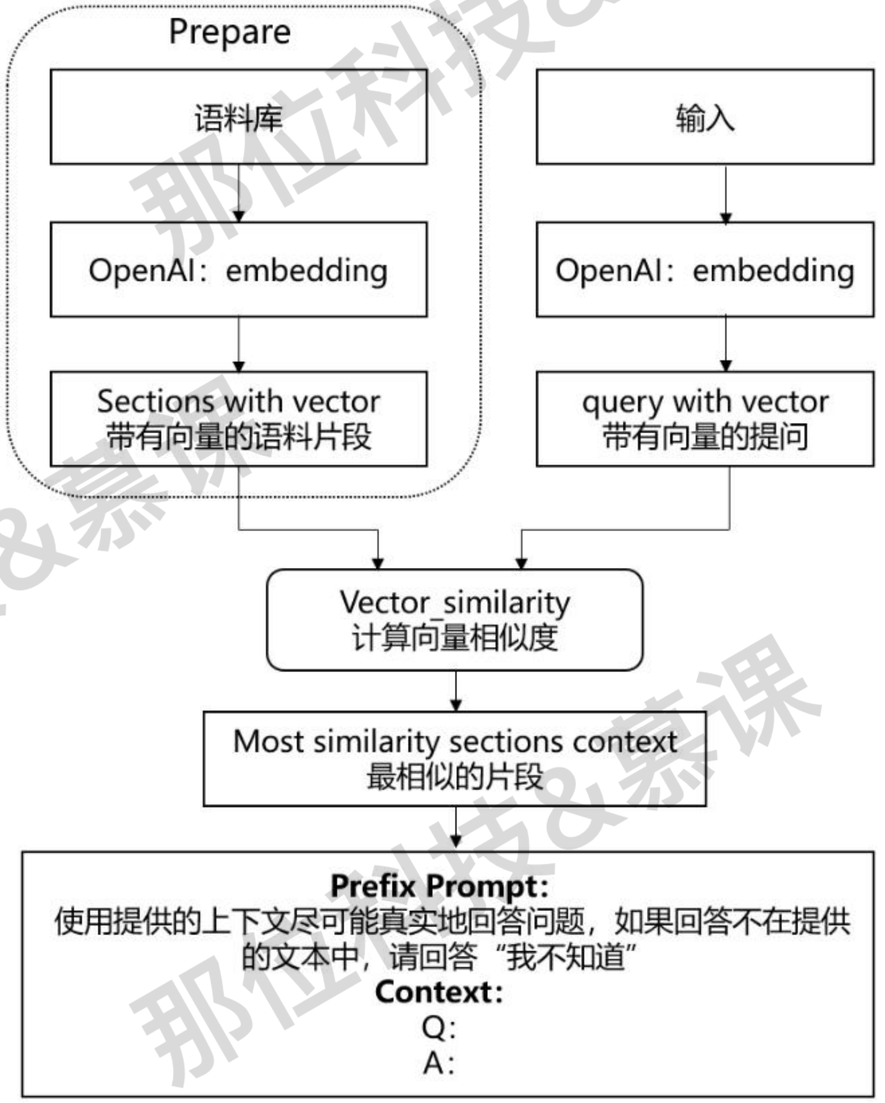
外挂文档问答的流程

大模型之家

➢**语料库准备**
  ➢将与行业相关的大量知识或语料上传至向量数据库，储存为向量化文本；
➢**问题输入**
  ➢输入的问题被Embedding引擎变成带有向量的提问；
➢**向量搜索**
  ➢向量化问题进入提前准备好的向量数据库中，通过向量搜
➢**计算向量相似度**
  ➢匹配出TopN条语义最相关的Facts(向量数据库是模糊匹配，输出的是概率上最近似的答案)

➢**Prompt优化**
  ➢输出的TopN条Facts，和用户的问题一起作为prompt输入给模型。

➢**结果返回**
  ➢有记忆交互下得到的生成内容更精准且缓解了幻觉问题。



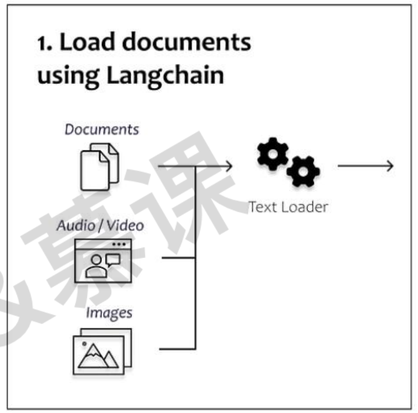
那位科技&慕课

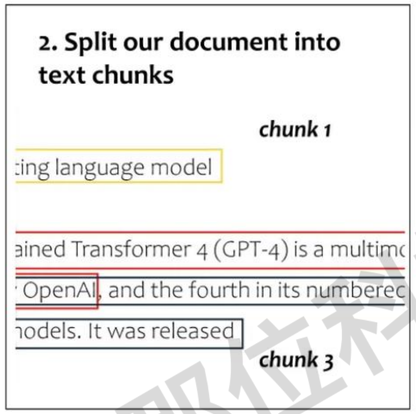
外挂文档问答的流程

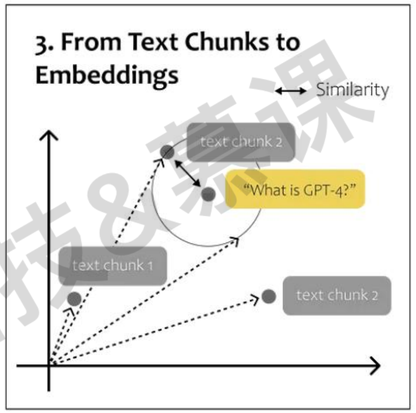
大模型之家

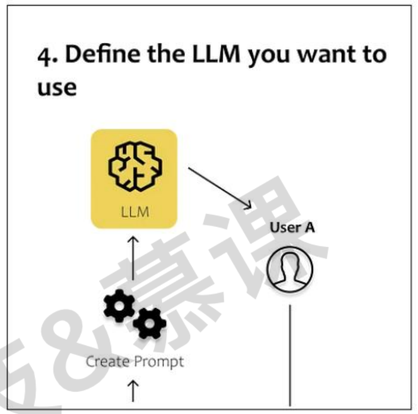
➢**主要思想**：Langchain先使用索引模块中的文档加载器和文本拆分器加载和处理非结构化数据。然后使用提示模块将找到的内容，这个过程使用的是相似度计算召回。之后，注入到提示模板中，并使用模型的模块将提示发送到模型完成生成。

➢**流程：加载文件 -> 读取文本 -> 文本分割 -> 文本向量化 -> 问句向量化 -> 在文本向量中匹配出与问句向量最相似的top k个 -> 匹配出的文本作为上下文和问题一起添加到prompt中 -> 提交给LLM生成回答。**

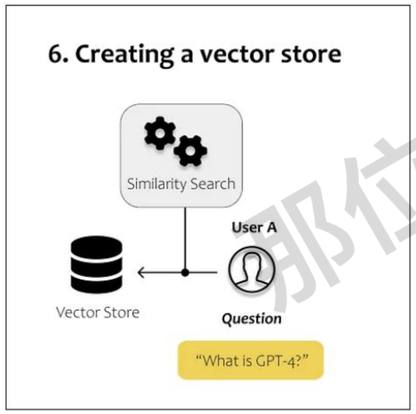












那位科技&慕课

外挂文档问答之文档解析

大模型之家

➢任务：为了完成针对文档的问答，需要对文档进行解析，而文档各异，涉及到pdf、html、epub等不同格式的文本

➢解决方案：针对不同的文档，内置不同的解析脚本，针对性地调用，最后都转换成txt纯文本格式

➢举例：针对pdf文件，内置PyPDFparser，pdfminer等。

langchain / langchain / document_loaders / parsers / pdf.py

lucasiscovici Add password to PyPDR loader and parser (#6908) e995039 · last week History

Code Blame 151 lines (122 loc) · 5.14 KB Raw

```python
"""Module contains common parsers for PDFs."""
from typing import Any, Iterator, Mapping, Optional, Union

from langchain.document_loaders.base import BaseBlobParser
from langchain.document_loaders.blob_loaders import Blob
from langchain.schema import Document


class PyPDFParser(BaseBlobParser):
    """Loads a PDF with pypdf and chunks at character level."""

    def __init__(self, password: Optional[Union[str, bytes]] = None):
        self.password = password

    def lazy_parse(self, blob: Blob) -> Iterator[Document]:
        """Lazily parse the blob."""
        import pypdf

        with blob.as_bytes_io() as pdf_file_obj:
            pdf_reader = pypdf.PdfReader(pdf_file_obj, password=self.password)
            yield from [
                Document(
                    page_content=page.extract_text(),
                    metadata={"source": blob.source, "page": page_number},
                )
                for page_number, page in enumerate(pdf_reader.pages)
            ]


class PDFMinerParser(BaseBlobParser):
    """Parse PDFs with PDFMiner."""

    def lazy_parse(self, blob: Blob) -> Iterator[Document]:
        """Lazily parse the blob."""
        from pdfminer.high_level import extract_text

        with blob.as_bytes_io() as pdf_file_obj:
            text = extract_text(pdf_file_obj)
            metadata = {"source": blob.source}
            yield Document(page_content=text, metadata=metadata)

```

langchain / langchain / document_loaders /

mhtml.py	docstrings document_loaders 2 (#6890)
modern_treasury.py	added docstrings where they missed (#6626)
notebook.py	changed ValueError to ImportError (#5103)
notion.py	Harrison/unstructured support (#903)
notiondb.py	Improve performance when retrieving Notion DB pages (#6710)
obsidian.py	Dev2049/obsidian patch (#4204)
odt.py	
onedrive.py	
onedrive_file.py	
open_city_data.py	
org_mode.py	
pdf.py	
powerpoint.py	
psychic.py	
pyspark_dataframe.py	
python.py	
readthedocs.py	
recursive_url_loader.py	
reddit.py	
roam.py	
rst.py	
rtf.py	
s3_directory.py	

```python
import requests
from bs4 import BeautifulSoup
from langchain.embeddings.openai import OpenAIEmbeddings
from langchain.text_splitter import CharacterTextSplitter
from langchain.vectorstores import Chroma
from langchain.document_loaders import TextLoader

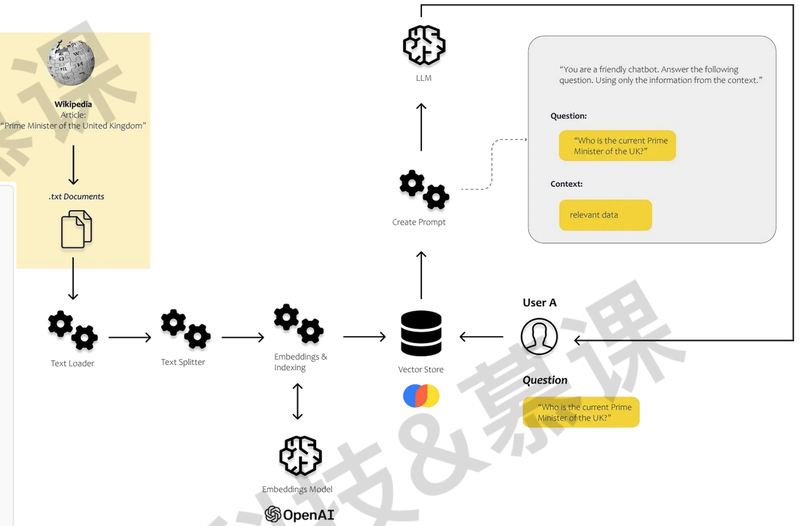
# URL of the Wikipedia page to scrape
url = 'https://en.wikipedia.org/wiki/Prime_Minister_of_the_United_Kingdom'

# Send a GET request to the URL
response = requests.get(url)

# Parse the HTML content using BeautifulSoup
soup = BeautifulSoup(response.content, 'html.parser')

# Find all the text on the page
text = soup.get_text()
text = text.replace('\n', '')

# Open a new file called 'output.txt' in write mode and store the file object in
with open('output.txt', 'w', encoding='utf-8') as file:
    # Write the string to the file
    file.write(text)
```



外挂文档问答之文档切分

大模型之家

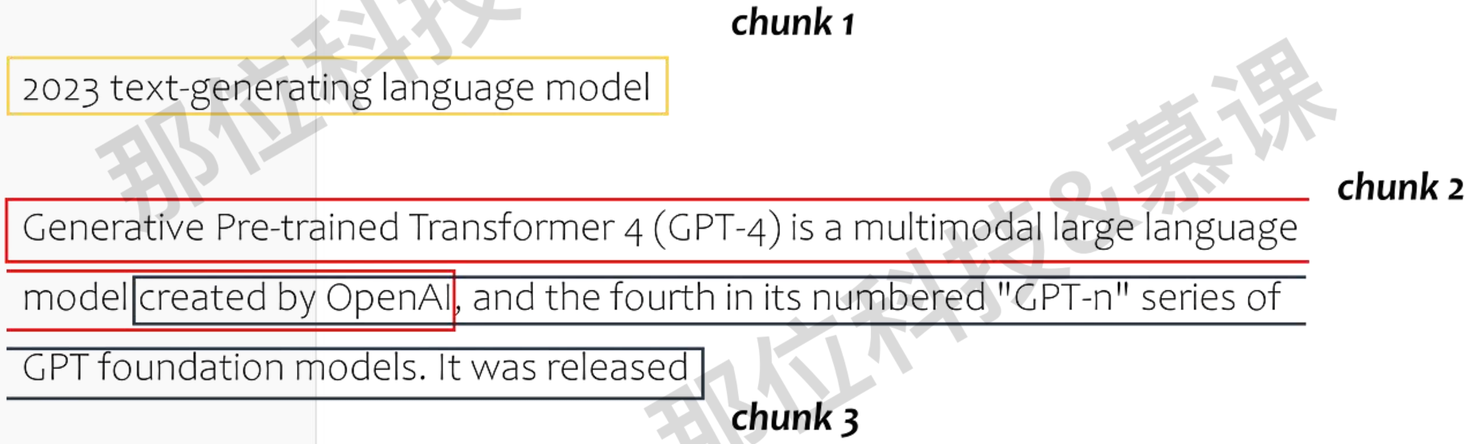
➢**任务**：在标准化文本之后，则需要对文本进行切分，即将文本划分为称为文本块的较小部分，
➢**langchain的拆分方法**：https://github.com/hwchase17/langchain/blob/master/langchain/text_splitter.py
➢**langchain的内置文本拆分模块设定了几个参数**：
  ➢**chunk_size**：文本块的大小，即文本块的最大尺寸；
  ➢**chunk_overlap**：表示两个切分文本之间的重合度，文本块之间的最大重叠量，保留一些重叠可以保持文本块之间的连续性，可以使用滑动窗口进行构造，这个比较重要。
  ➢**length_function**：用于计算文本块长度的方法，默认为简单的计算字符数；

```python
from langchain.text_splitter import RecursiveCharacterTextSplitter

# load the document
with open('./output.txt', encoding='utf-8') as f:
    text = f.read()

# define the text splitter
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size = 500,
    chunk_overlap  = 100,
    length_function = len,
)

texts = text_splitter.create_documents([text])
```

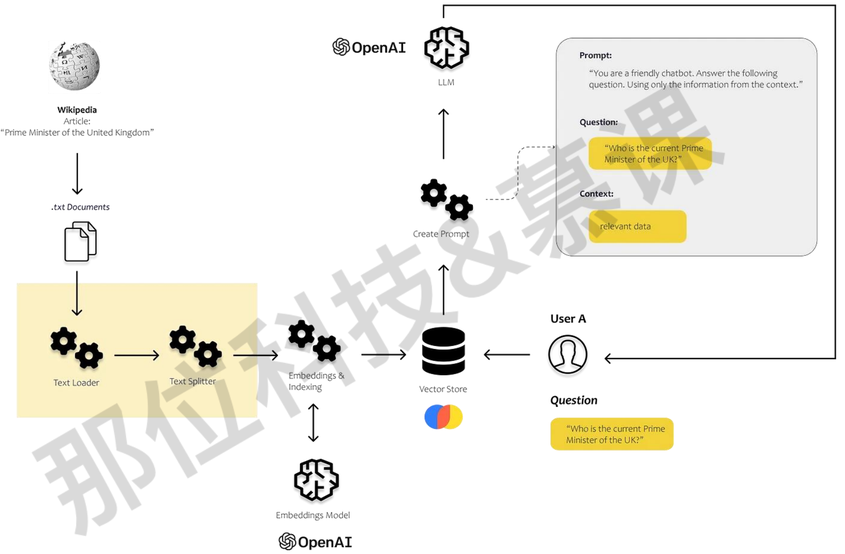


外挂文档问答之文档切分

大模型之家

➢**任务**：在标准化文本之后，则需要对文本进行切分，即将文本划分为称为文本块的较小部分，
➢**langchain的拆分方法**：https://github.com/hwchase17/langchain/blob/master/langchain/text_splitter.py
➢**langchain的内置文本拆分模块设定了几个参数**：
  ➢**chunk_size**：文本块的大小，即文本块的最大尺寸；
  ➢**chunk_overlap**：表示两个切分文本之间的重合度，文本块之间的最大重叠量，保留一些重叠可以保持文本块之间的连续性，可以使用滑动窗口进行构造，这个比较重要。
  ➢**length_function**：用于计算文本块长度的方法，默认为简单的计算字符数；

文本分割器	
LatexTextSplitter	沿着Latex标题、标题、枚举等分割文本。
MarkdownTextSplitter	沿着Markdown的标题、代码块或水平规则来分割文本。
NLTKTextSplitter	使用NLTK的分割器
PythonCodeTextSplitter	沿着Python类和方法的定义分割文本。
RecursiveCharacterTextSplitter	用于通用文本的分割器。它以一个字符列表为参数，尽可能地把所有的段落（然后是句子，然后是单词）放在一起
SpacyTextSplitter	使用Spacy的分割器
TokenTextSplitter	根据openAI的token数进行分割



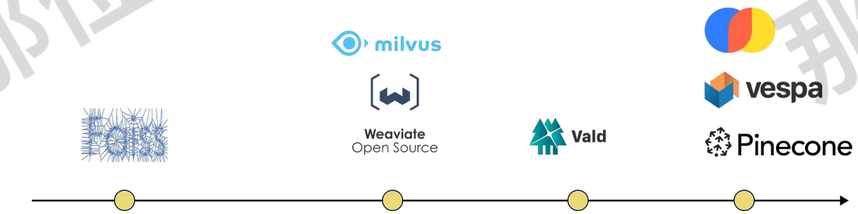
那位科技&慕课
老刘说NLP

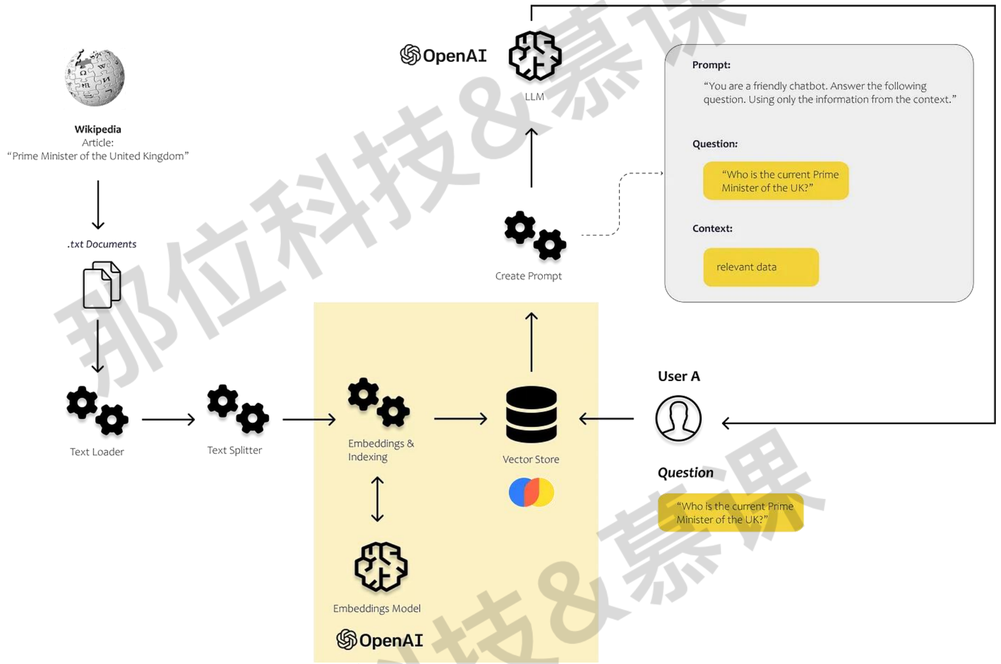
外挂文档问答之文档向量化

大模型之家

➢任务：在完成文本的分割之后，由于后期需要进行搜索，因此需要将文本进行向量化表示，将文本映射为低维稠密的向量化表示

➢方案：默认使用Openai—text—embdding—ada—002嵌入表示(本身为一个向量化模型)






```python
from langchain.embeddings.openai import OpenAIEmbeddings
from langchain.vectorstores import Chroma

# define the embeddings model
embeddings = OpenAIEmbeddings()

# use the text chunks and the embeddings model to fill our vector store
db = Chroma.from_documents(texts, embeddings)
```

```python
import openai

print(texts[0])

embedding = openai.Embedding.create(
    input=texts[0].page_content, model="text-embedding-ada-002"
)["data"][0]["embedding"]

len(embedding)
```

[13] ✓ 1.3s

```
page_content='2023 text-generating language model' metadata={}

1536
```

```python
embedding
```

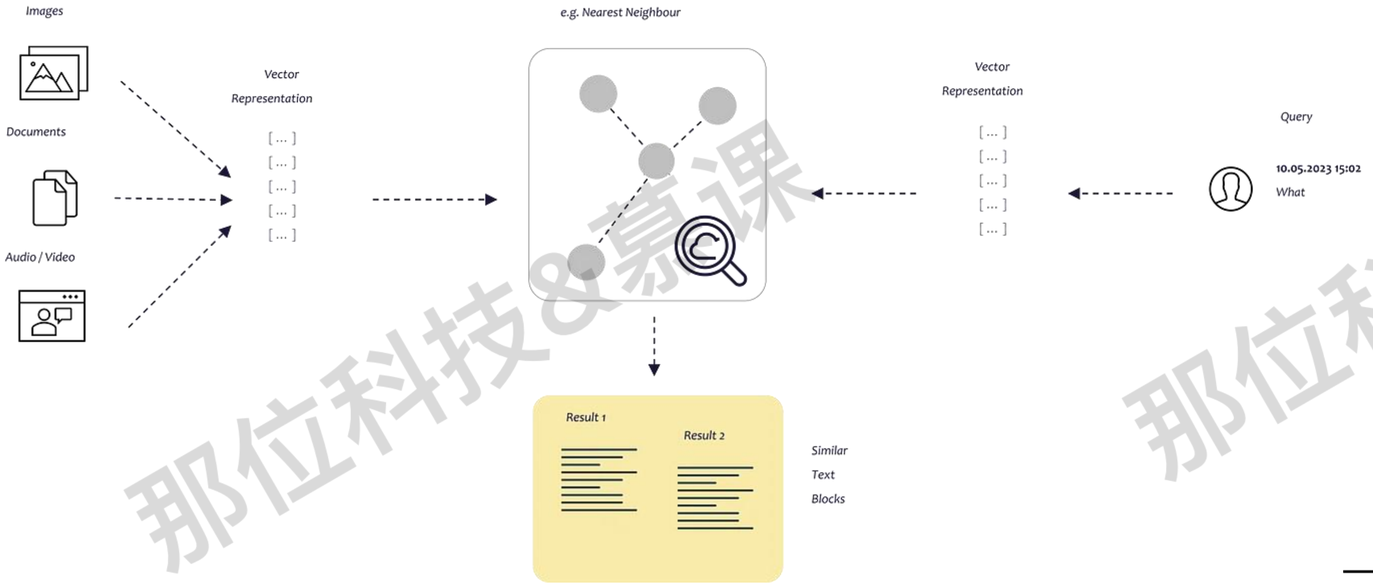
[14] ✓ 0.0s

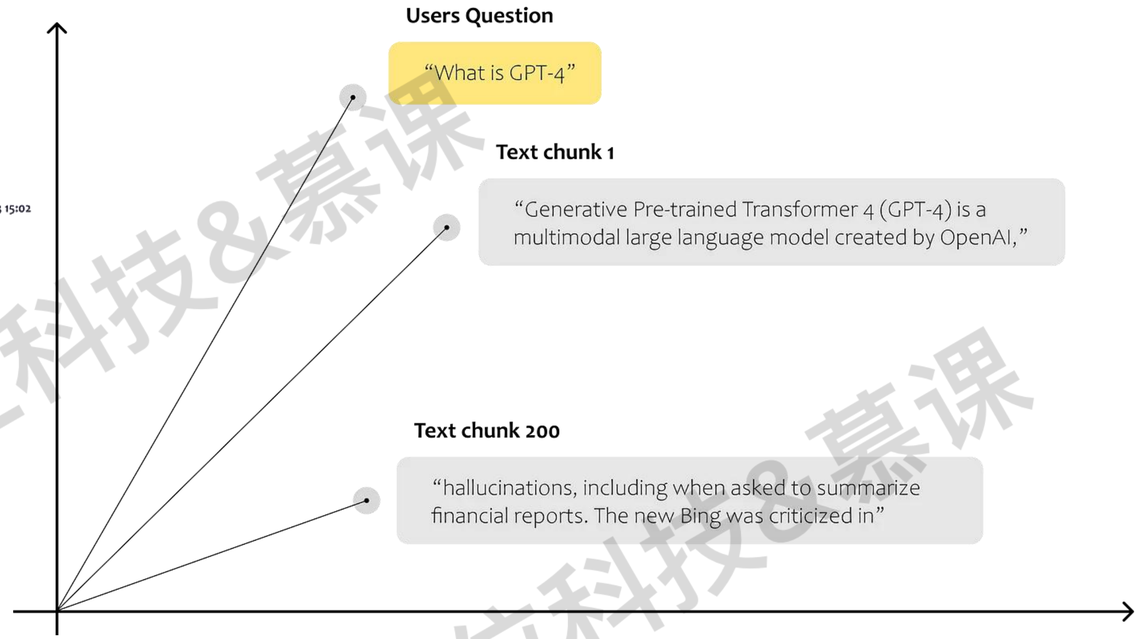
```
[-0.03264317288994789,
 0.0002940905396826565,
 -0.005302408244460821,
 0.019748838618397713,
 0.010710163041949272,
```

外挂文档问答之文本召回

大模型之家

➢**任务**：在完成文本的向量化之后，需要完成用户query到本地知识库两者之间的相似度召回
➢**方案**：默认使用cosine similarity，即余弦相似度，计算相似度后按照从大到小排序





那位科技&慕课

外挂文档问答之文本召回

大模型之家

➢**任务**：在完成文本的向量化之后，需要完成用户query到本地知识库两者之间的相似度召回
➢**方案**：默认使用cosine similarity，即余弦相似度，计算相似度后按照从大到小排序

```
import numpy as np
from numpy.linalg import norm


# calculate the embeddings for the user's question
users_question = "What is GPT-4?"
question_embedding = get_embedding(text=users_question, model="text-embedding-ad


# create a list to store the calculated cosine similarity
cos_sim = []


for index, row in df.iterrows():
   A = row.ada_embedding
   B = question_embedding


   # calculate the cosine similarity
   cosine = np.dot(A,B)/(norm(A)*norm(B))


   cos_sim.append(cosine)


df["cos_sim"] = cos_sim
df.sort_values(by=["cos_sim"], ascending=False)
```

	text_chunks	ada_embedding	cos_sim
11	previous (GPT-3.5 based) ChatGPT, with the cav...	[0.0009534807177260518, -0.0003470357623882591...	0.875788
26	Unlike its predecessors, GPT-4 is a multimodal...	[-0.02740379050374031, 0.011374002322554588, -...	0.871286
1	Generative Pre-trained Transformer 4 (GPT-4) i...	[-0.019854722544550896, -0.03438127413392067, ...	0.866611
10	Observers reported the GPT-4 based version of ...	[-0.006986829452216625, 0.008246184326708317, ...	0.861069
14	Further information: GPT-3 § Background, and G...	[0.0011879028752446175, -0.006450972054153681,...	0.856213
...	...	...	...
154	Luccioni, a research scientist at HuggingFace,...	[-0.008462309837341309, 0.007041186094284058, ...	0.675826
74	to harm oneself or others, or requests for des...	[-0.0021866238676011562, 0.01054801233112812, ...	0.675248
194	Microsoft says that waitlisted users will be p...	[-0.019956298172473907, -0.006524444557726383,...	0.671096
102	U.S. Representatives Don Beyer and Ted Lieu co...	[0.01705823466181755, -0.02539057843387127, -0...	0.670930

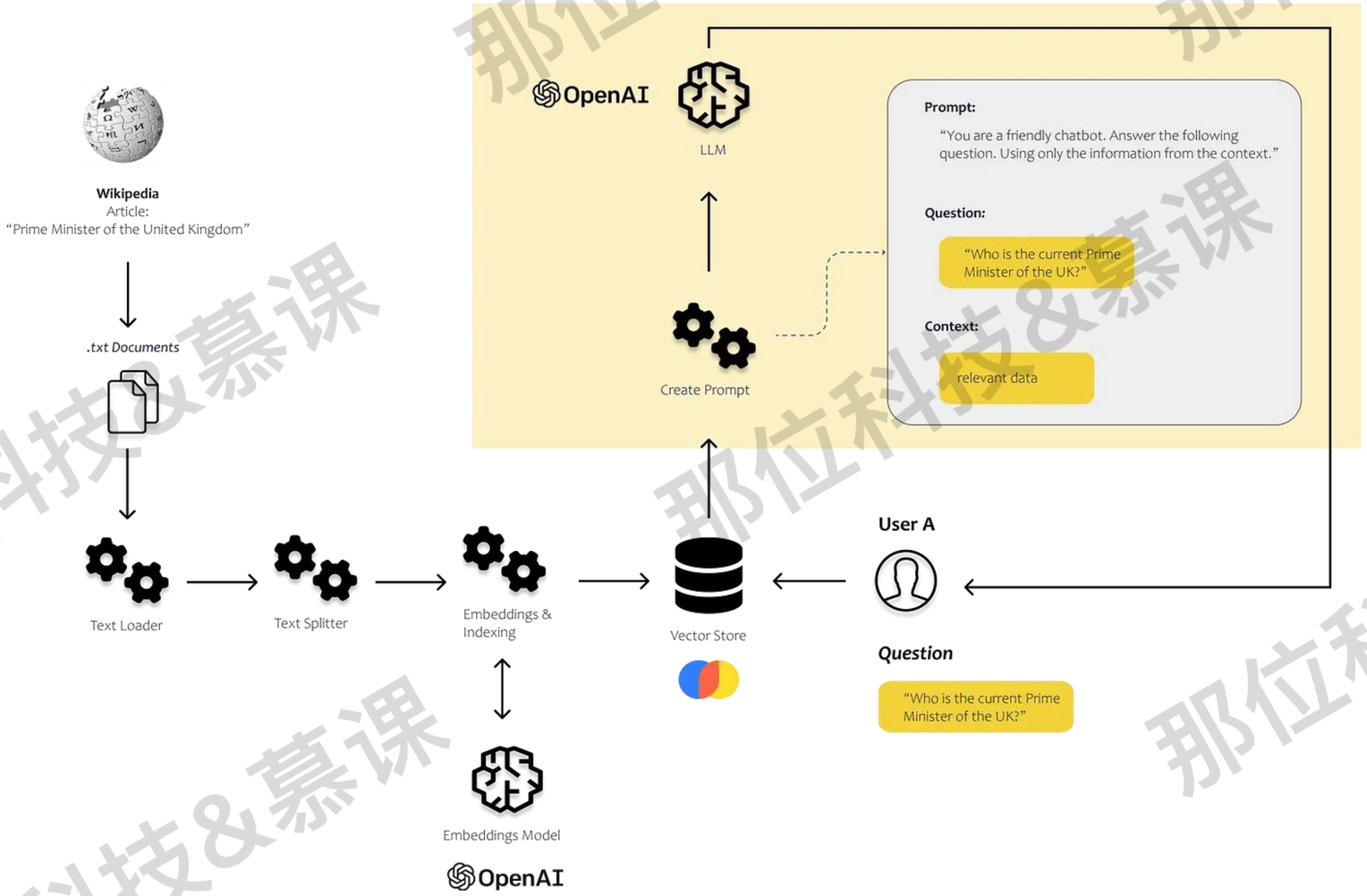
那位科技&慕课

外挂文档问答之prompt设计

大模型之家

➢**任务**：**在完成文本向量化召回后，已经获得了与用户输入query的相关上下文**，需要借助生成模型进行回复生成

➢**方案**：设计prompt，将用户query与召回结果进行组合拼接，然后送入模型完成生成。



那位科技&慕课

外挂文档问答之prompt设计

大模型之家

➢**任务**：**在完成文本向量化召回后，已经获得了与用户输入query的相关上下文**，需要借助生成模型进行回复生成

➢**方案**：设计prompt，将用户query与召回结果进行组合拼接，然后送入模型完成生成。

```
from langchain.llms import OpenAI
from langchain import PromptTemplate

users_question = "Who is the current Prime Minister of the UK?"

# use our vector store to find similar text chunks
results = db.similarity_search(
    query=user_question,
    n_results=5
)

# define the prompt template
template = """
You are a chat bot who loves to help people! Given the following context sections,
question using only the given context. If you are unsure and the answer is not
explicitly writting in the documentation, say "Sorry, I don't know how to help wit

Context sections:
{context}

Question:
{users_question}

Answer:
"""

prompt = PromptTemplate(template=template, input_variables=["context", "users_ques

# fill the prompt template
prompt_text = prompt.format(context = results, users_question = users_question)

# ask the defined LLM
llm(prompt_text)
```

```
    llm(prompt_text)
[197]  ✓ 1.5s
...  'The current Prime Minister of the UK is Rishi Sunak, since 25 October 2022.'
```

那位科技&慕课

大模型文档问答的几个小案例

案例1：版面分析

大模型之家

➢**地址**：https://github.com/PaddlePaddle/PaddleOCR/blob/release/2.6/ppstructure/docs/quickstart.md
➢开源版本效果实测并不是太好，有条件的可以自行进行训练

PP-Structure 快速开始

- 1. 准备环境
- 2. 便捷使用
  - 2.1 命令行使用
    - 2.1.1 图像方向分类+版面分析+表格识别
    - 2.1.2 版面分析+表格识别
    - 2.1.3 版面分析
    - 2.1.4 表格识别
    - 2.1.5 关键信息抽取
    - 2.1.6 版面恢复
  - 2.2 Python脚本使用
    - 2.2.1 图像方向分类+版面分析+表格识别
    - 2.2.2 版面分析+表格识别
    - 2.2.3 版面分析
    - 2.2.4 表格识别
    - 2.2.5 关键信息抽取
    - 2.2.6 版面恢复
  - 2.3 返回结果说明
    - 2.3.1 版面分析+表格识别
    - 2.3.2 关键信息抽取
  - 2.4 参数说明
- 3. 小结

1. 准备环境

1.1 安装PaddlePaddle

> 如果您没有基础的Python运行环境，请参考运行环境准备。

- 您的机器安装的是CUDA9或CUDA10，请运行以下命令安装

```
python3 -m pip install paddlepaddle-gpu -i https://mirror.baidu.com/pypi/simple
```

- 您的机器是CPU，请运行以下命令安装

```
python3 -m pip install paddlepaddle -i https://mirror.baidu.com/pypi/simple
```

那位科技&慕课

案例1：版面分析

大模型之家

➢**地址**：https://github.com/PaddlePaddle/PaddleOCR/blob/release/2.6/ppstructure/docs/quickstart.md
➢开源版本效果实测并不是太好，有条件的可以自行进行训练

```python
import os
import cv2
from paddleocr import PPStructure,draw_structure_result,save_structure_res

table_engine = PPStructure(show_log=True, image_orientation=True)

save_folder = './output'
img_path = 'ppstructure/docs/table/1.png'
img = cv2.imread(img_path)
result = table_engine(img)
save_structure_res(result, save_folder,os.path.basename(img_path).split('.')[0])

for line in result:
    line.pop('img')
    print(line)

from PIL import Image

font_path = 'doc/fonts/simfang.ttf' # PaddleOCR下提供字体包
image = Image.open(img_path).convert('RGB')
im_show = draw_structure_result(image, result,font_path=font_path)
im_show = Image.fromarray(im_show)
im_show.save('result.jpg')
```

```
[
  {   'type': 'Text',
      'bbox': [34, 432, 345, 462],
      'res': ([[36.0, 437.0, 341.0, 437.0, 341.0, 446.0, 36.0, 447.0], [41.0, 454.0, 125.0,
      453.0, 125.0, 459.0, 41.0, 460.0]],
                 [('Tigure-6. The performance of CNN and IPT models using differen', 0.90060663),
                 ('Tent  ', 0.465441)])
  }
]
```

字段	说明
type	图片区域的类型
bbox	图片区域的在原图的坐标，分别[左上角x，左上角y，右下角x，右下角y]
res	图片区域的OCR或表格识别结果。 表格: 一个dict，字段说明如下 `html`：表格的HTML字符串 在代码使用模式下，前向传入return_ocr_result_in_table=True可以拿到表格中每个文本的检测识别结果，对应为如下字段: `boxes`：文本检测坐标 `rec_res`：文本识别结果。 OCR: 一个包含各个单行文字的检测坐标和识别结果的元组

那位科技&慕课

案例2：文本切分

大模型之家

➢BERT语义文档切分

➢地址：https://github.com/chatchat-space/Langchain-Chatchat/blob/master/text_splitter/chinese_recursive_text_splitter.py

```python
class RapidOCRPDFLoader(UnstructuredFileLoader):
    def _get_elements(self) -> List:
        def pdf2text(filepath):
            import fitz # pyMuPDF里面的fitz包，不要与pip install fitz混淆
            from rapidocr_onnxruntime import RapidOCR
            import numpy as np
            ocr = RapidOCR()
            doc = fitz.open(filepath)
            resp = ""

            b_unit = tqdm.tqdm(total=doc.page_count, desc="RapidOCRPDFLoader context page index: 0")
            for i, page in enumerate(doc):

                # 更新描述
                b_unit.set_description("RapidOCRPDFLoader context page index: {}".format(i))
                # 立即显示进度条更新结果
                b_unit.refresh()
                # TODO: 依据文本与图片顺序调整处理方式
                text = page.get_text("")
                resp += text + "\n"

                img_list = page.get_images()
                for img in img_list:
                    pix = fitz.Pixmap(doc, img[0])
                    img_array = np.frombuffer(pix.samples, dtype=np.uint8).reshape(pix.height, pix.width, -1)
                    result, _ = ocr(img_array)
                    if result:
                        ocr_result = [line[1] for line in result]
                        resp += "\n".join(ocr_result)

                # 更新进度
                b_unit.update(1)
            return resp

        text = pdf2text(self.file_path)
        from unstructured.partition.text import partition_text
        return partition_text(text=text, **self.unstructured_kwargs)
```

```python
def split_text(self, text: str) -> List[str]:   ##此处需要进一步优化逻辑
    if self.pdf:
        text = re.sub(r"\n{3,}", r"\n", text)
        text = re.sub('\s', " ", text)
        text = re.sub("\n\n", "", text)

    text = re.sub(r'([;；.!?。！？\?])([^”’])', r"\1\n\2", text)  # 单字符断句符
    text = re.sub(r'(\.{6})([^"’”」』])', r"\1\n\2", text)  # 英文省略号
    text = re.sub(r'(\…{2})([^"’”」』])', r"\1\n\2", text)  # 中文省略号
    text = re.sub(r'([;；!?。！？\?]["’”」』]{0,2})([^;；!?，。！？\?])', r'\1\n\2', text)
    # 如果双引号前有终止符，那么双引号才是句子的终点，把分句符\n放到双引号后，注意前面的几句都小心保留了双引号
    text = text.rstrip()  # 段尾如果有多余的\n就去掉它
    # 很多规则中会考虑分号；，但是这里我把它忽略不计，破折号、英文双引号等同样忽略，需要的再做些简单调整即可。
    ls = [i for i in text.split("\n") if i]
    for ele in ls:
        if len(ele) > self.sentence_size:
            ele1 = re.sub(r'([,，.]["’”」』]{0,2})([^,，.])', r'\1\n\2', ele)
            ele1_ls = ele1.split("\n")
            for ele_ele1 in ele1_ls:
                if len(ele_ele1) > self.sentence_size:
                    ele_ele2 = re.sub(r'([\n]{1,}| {2,}["’”」』]{0,2})([^\s])', r'\1\n\2', ele_ele1)
                    ele2_ls = ele_ele2.split("\n")
                    for ele_ele2 in ele2_ls:
                        if len(ele_ele2) > self.sentence_size:
                            ele_ele3 = re.sub('( ["’”」』]{0,2})([^ ])', r'\1\n\2', ele_ele2)
                            ele2_id = ele2_ls.index(ele_ele2)
                            ele2_ls = ele2_ls[:ele2_id] + [i for i in ele_ele3.split("\n") if i] + ele2_ls[
                                                                                                  ele2_id + 1:]
                    ele_id = ele1_ls.index(ele_ele1)
                    ele1_ls = ele1_ls[:ele_id] + [i for i in ele2_ls if i] + ele1_ls[ele_id + 1:]

            id = ls.index(ele)
            ls = ls[:id] + [i for i in ele1_ls if i] + ls[id + 1:]
    return ls
```

那位科技&慕课

案例2：文本切分

大模型之家

➢BERT语义文档切分

➢地址：https://www.modelscope.cn/models/damo/nlp_bert_document-segmentation_chinese-base/summary

```
from modelscope.outputs import OutputKeys
from modelscope.pipelines import pipeline
from modelscope.utils.constant import Tasks

p = pipeline(
    task=Tasks.document_segmentation,
    model='damo/nlp_bert_document-segmentation_chinese-base')

result = p(documents='移动端语音唤醒模型，检测关键词为“小云小云”。模型主体为4层FSMN结构，使用CTC训练准则，参数量750K，适用于移动端设备运行。模型输入为Fbank特征，输出为基于char建模的中文全集token预测，测试工具根据每一帧的预测数据进行后处理得到输入音频的实时检测结果。模型训练采用“basetrain + finetune”的模式，basetrain过程使用大量内部移动端数据，在此基础上，使用1万条设备端录制安静场景“小云小云”数据进行微调，得到最终面向业务的模型。后续用户可在basetrain模型基础上，使用其他关键词数据进行微调，得到新的语音唤醒模型，但暂时未开放模型finetune功能。')

print(result[OutputKeys.TEXT])
```

那位科技&慕课

案例2：文本切分

大模型之家

➢地址：https://www.modelscope.cn/models/damo/nlp_bert_document-segmentation_chinese-base/summary

测试内容 更换示例

近年来，随着端到端语音识别的流行，基于Transformer结构的语音识别系统逐渐成为了主流。然而，由于Transformer是一种自回归模型，需要逐个生成目标文字，计算复杂度随着目标文字数量线性增加，限制了其在工业生产中的应用。针对Transoformer模型自回归生成文字的低计算效率缺陷，学术界提出了非自回归模型来并行的输出目标文字。根据生成目标文字时，迭代轮数，非自回归模型分为：多轮迭代式与单轮迭代非自回归模型。其中实用的是基于单轮迭代的>非自回归模型。对于单轮非自回归模型，现有工作往往聚焦于如何更加准确的预测目标文字个数，如CTC-enhanced采用CTC预测输出文字个数，尽管如此，考虑到现实应用中，语速、口音、静音以及噪声等因素的影响，如何准确的预测目标文字个数以及抽取目标文字对应的声学隐变量仍然是一个比较大的挑战；另外一方面，我们通过对比自回归模型与单轮非自回归模型在工业大数据上的错误类型（如下图所示，AR与vanilla NAR），发现，相比于自回归模型，非自回归模型，在预测目标文字个数方面差距较小，但是替换错误显著的增加，我们认为这是由于单轮非自回归模型中条件独立假设导致的语义信息丢失。于此同时，目前非自回归模型主要停留在学术验证阶段，还没有工业大数据上的相关实验与结论。

单词数: 461 / 1000

执行测试

测试结果 模型推理耗时 0.06s

JSO jsonOutput

近年来，随着端到端语音识别的流行，基于Transformer结构的语音识别系统逐渐成为了主流。

然而，由于Transformer是一种自回归模型，需要逐个生成目标文字，计算复杂度随着目标文字数量线性增加，限制了其在工业生产中的应用。针对Transoformer模型自回归生成文字的低计算效率缺陷，学术界提出了非自回归模型来并行的输出目标文字。根据生成目标文字时，迭代轮数，非自回归模型分为：多轮迭代式与单轮迭代非自回归模型。其中实用的是基于单轮迭代的>非自回归模型。对于单轮非自回归模型，现有工作往往聚焦于如何更加准确的预测目标文字个数，如CTC-enhanced采用CTC预测输出文字个数，尽管如此，考虑到现实应用中，语速、口音、静音以及噪声等因素的影响，如何准确的预测目标文字个数以及抽取目标文字对应的声学隐变量仍然是一个比较大的挑战；另外一方面，我们通过对比自回归模型与单轮非自回归模型在工业大数据上的错误类型（如下图所示，AR与vanilla NAR），发现，相比于自回归模型，非自回归模型，在预测目标文字个数方面差距较小，但是替换错误显著的增加，我们认为这是由于单轮非自回归模型中条件独立假设导致的语义信息丢失。于此同时，目前非自回归模型主要停留在学术验证阶段，还没

那位科技&慕课

案例3：text2vec向量化

大模型之家

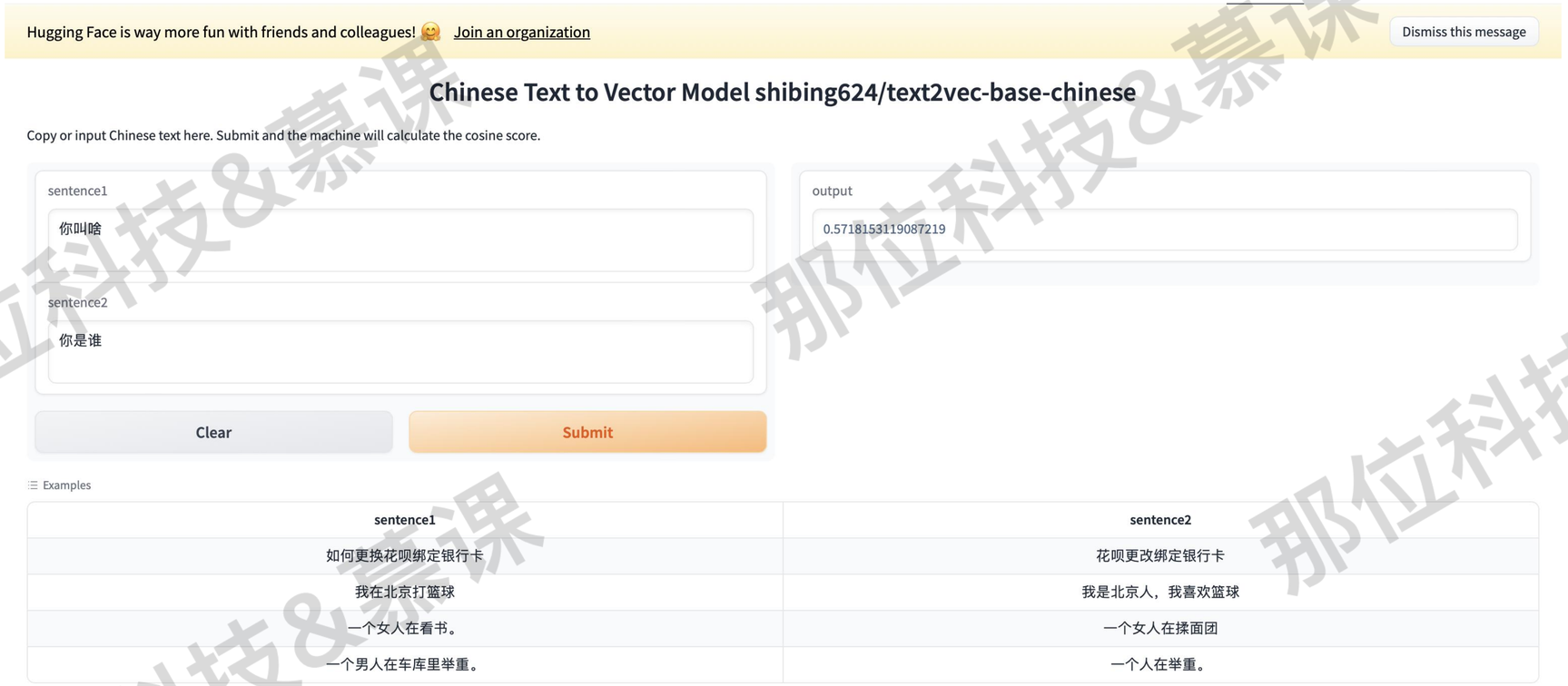
➢**地址**：https://github.com/shibing624/text2vec

➢Text2vec：Text to Vector，Get Sentence Embeddings，文本向量化，把文本(包括词、句子、段落)表征为向量矩阵。

➢text2vec实现了Word2Vec、RankBM25、BERT、Sentence-BERT、CoSENT等多种文本表征、文本相似度计算模型，并在文本语义匹配（相似度计算）任务上比较了各模型的效果。

➢Official Demo：https://www.mulanai.com/product/short_text_sim/

➢HuggingFace Demo：https://huggingface.co/spaces/shibing624/text2vec



那位科技&慕课

案例3：text2vec向量化

大模型之家

➢https://github.com/shibing624/text2vec/blob/master/examples/computing_embeddings_demo.py

```python
import sys

sys.path.append('..')
from text2vec import SentenceModel
from text2vec import Word2Vec


def compute_emb(model):
    # Embed a list of sentences
    sentences = [
        '卡',
        '银行卡',
        '如何更换花呗绑定银行卡',
        '花呗更改绑定银行卡',
        'This framework generates embeddings for each input sentence',
        'Sentences are passed as a list of string.',
        'The quick brown fox jumps over the lazy dog.'
    ]
    sentence_embeddings = model.encode(sentences)
    print(type(sentence_embeddings), sentence_embeddings.shape)

    # The result is a list of sentence embeddings as numpy arrays
    for sentence, embedding in zip(sentences, sentence_embeddings):
        print("Sentence:", sentence)
        print("Embedding shape:", embedding.shape)
        print("Embedding head:", embedding[:10])
        print()


if __name__ == "__main__":
    # 中文句向量模型(CoSENT)，中文语义匹配任务推荐，支持fine-tune继续训练
    t2v_model = SentenceModel("shibing624/text2vec-base-chinese")
    compute_emb(t2v_model)

    # 支持多语言的句向量模型（CoSENT），多语言（包括中英文）语义匹配任务推荐，支持fine-tune继续训练
    sbert_model = SentenceModel("shibing624/text2vec-base-multilingual")
    compute_emb(sbert_model)

    # 中文词向量模型(word2vec)，中文字面匹配任务和冷启动适用
    w2v_model = Word2Vec("w2v-light-tencent-chinese")
    compute_emb(w2v_model)
```

Usage (sentence-transformers)

sentence-transformers is a popular library to compute dense vector representations for sentences.

Install sentence-transformers:

```
pip install -U sentence-transformers
```

Then load model and predict:

```python
from sentence_transformers import SentenceTransformer

m = SentenceTransformer("shibing624/text2vec-base-chinese")
sentences = ['如何更换花呗绑定银行卡', '花呗更改绑定银行卡']

sentence_embeddings = m.encode(sentences)
print("Sentence embeddings:")
print(sentence_embeddings)
```

那位科技&慕课

案例4：text2vec相似度检索

大模型之家

➢https://github.com/shibing624/text2vec/blob/master/examples/semantic_search_demo.py

```python
import sys

sys.path.append('..')
from text2vec import SentenceModel, cos_sim, semantic_search

embedder = SentenceModel()

# Corpus with example sentences
corpus = [
    '花呗更改绑定银行卡',
    '我什么时候开通了花呗',
    'A man is eating food.',
    'A man is eating a piece of bread.',
    'The girl is carrying a baby.',
    'A man is riding a horse.',
    'A woman is playing violin.',
    'Two men pushed carts through the woods.',
    'A man is riding a white horse on an enclosed ground.',
    'A monkey is playing drums.',
    'A cheetah is running behind its prey.'
]
corpus_embeddings = embedder.encode(corpus)

# Query sentences:
queries = [
    '如何更换花呗绑定银行卡',
    'A man is eating pasta.',
    'Someone in a gorilla costume is playing a set of drums.',
    'A cheetah chases prey on across a field.']

for query in queries:
    query_embedding = embedder.encode(query)
    hits = semantic_search(query_embedding, corpus_embeddings, top_k=5)
    print("\n\n======================\n\n")
    print("Query:", query)
    print("\nTop 5 most similar sentences in corpus:")
    hits = hits[0]  # Get the hits for the first query
    for hit in hits:
        print(corpus[hit['corpus_id']], "(Score: {:.4f})".format(hit['score']))
```

output:

```
Query: 如何更换花呗绑定银行卡
Top 5 most similar sentences in corpus:
花呗更改绑定银行卡 (Score: 0.9477)
我什么时候开通了花呗 (Score: 0.3635)
A man is eating food. (Score: 0.0321)
A man is riding a horse. (Score: 0.0228)
Two men pushed carts through the woods. (Score: 0.0090)


======================
Query: A man is eating pasta.
Top 5 most similar sentences in corpus:
A man is eating food. (Score: 0.6734)
A man is eating a piece of bread. (Score: 0.4269)
A man is riding a horse. (Score: 0.2086)
A man is riding a white horse on an enclosed ground. (Score: 0.1020)
A cheetah is running behind its prey. (Score: 0.0566)


======================
Query: Someone in a gorilla costume is playing a set of drums.
Top 5 most similar sentences in corpus:
A monkey is playing drums. (Score: 0.8167)
A cheetah is running behind its prey. (Score: 0.2720)
A woman is playing violin. (Score: 0.1721)
A man is riding a horse. (Score: 0.1291)
A man is riding a white horse on an enclosed ground. (Score: 0.1213)


======================
Query: A cheetah chases prey on across a field.
Top 5 most similar sentences in corpus:
A cheetah is running behind its prey. (Score: 0.9147)
A monkey is playing drums. (Score: 0.2655)
A man is riding a horse. (Score: 0.1933)
A man is riding a white horse on an enclosed ground. (Score: 0.1733)
A man is eating food. (Score: 0.0329)
```

那位科技&慕课

案例5：faiss向量数据库

大模型之家

➢https://github.com/facebookresearch/faiss/wiki/Getting-started

```
pip install faiss-cpu
```

```
pip install faiss-gpu
```

```
import numpy as np
import faiss
```

```
d = 128                           # dimension
nb = 10000                        # database size
np.random.seed(1234)              # make reproducible
xb = np.random.random((nb, d)).astype('float32')
```

那位科技&慕课

案例5：faiss向量数据库

大模型之家

➢https://github.com/facebookresearch/faiss/wiki/Getting-started

```
index = faiss.IndexFlatL2(d)   # build the index
print(index.is_trained)
```

```
index.add(xb)                  # add vectors to the index
print(index.ntotal)
```

```
nq = 5                         # number of query vectors
k = 4                          # we want 4 similar vectors
Xq = np.random.random((nq, d)).astype('float32')
D, I = index.search(Xq, k)     # sanity check
print(I)
print(D)
```

那位科技&慕课

案例6：外挂实时搜索问答

大模型之家

➢https://mp.weixin.qq.com/s/77IY9ITSaC_SwdwyLiN17A

```
# coding = utf-8
import os
from duckduckgo_search import ddg
import json
import requests
import urllib3

def inference_chatglm6b(query):
    url = "http://【你的ip】/chat_with_history"
    data = json.dumps({"input_text": query})
    headers = {'Content-Type': 'application/xml'}
    try:
        res = requests.post(url, data=data, headers=headers)
        predict = json.loads(res.text)["response"]
    except Exception as e:
        print(e)
        predict = ''
    return predict

def search_web(query):
    search_results = ddg(query, max_results=10)
    content = set()
    for idx, result in enumerate(search_results):
        content.add(result["title"] + '\n' + result["body"])
    return '\n'.join(content)

def chat_web(query):
    print("query:", query)
    content = search_web(query)
    prompt = "给定以下内容：{web_content}回答问题， 问：{query}。\n回答：".format(web_content=content, query=query)
    answer = inference_chatglm6b(content)
    print("answer:", answer)
    print('******'*10)

if __name__ == '__main__':
    while 1:
        query = input("enter an query to search:").strip()
        chat_web(query)
```

```
**********************************************************
enter an query to search:马克龙在什么时候来北京了最近
query: 马克龙在什么时候来北京了最近
answer: 马克龙于 2023 年 4 月 5 日至 7 日对中国进行国事访问。
**********************************************************
enter an query to search:谁最近停售了张继科肖像产品
query: 谁最近停售了张继科肖像产品
answer: 蝴蝶乒乓球用品公司最近停售了张继科肖像产品。
**********************************************************
enter an query to search:特朗普认罪了么，最高会判几年
query: 特朗普认罪了么，最高会判几年
answer: 特朗普没有认罪，并且这些指控可能会导致他被判处136年的监禁。
**********************************************************
enter an query to search:张继科最近犯了啥事
query: 张继科最近犯了啥事
answer: 张继科最近犯了一些法律事件，其中包括恋爱期间劈腿、涉赌、泄露他人隐私视频以及欠下巨额债务，其中债主
**********************************************************
enter an query to search:北京最新天气
query: 北京最新天气
answer: 根据最新的天气预报，北京最新天气为多云转晴，最高气温为12°C，最低气温为3°C，风力较小，只有2、3级
**********************************************************
enter an query to search:今天几号
query: 今天几号
answer: 今天是 2023 年 4 月 4 日，是农历闰二月十四，同时也是阳历的星期四。
**********************************************************
enter an query to search:当前黄金价格多少
query: 当前黄金价格多少
answer: 根据提供的信息，今天是2023年4月4日，中国上海黄金交易所的黄金价格是439.6元人民币/克。这是裸黄金
**********************************************************
enter an query to search:马克龙访华，有哪三点值得关注
query: 马克龙访华，有哪三点值得关注
answer: 马克龙访华，值得关注的三点包括：
1. 中法合作新成果：这是马克龙首次访华，也是中法两国元首继去年G20巴厘岛峰会后再一次面对面对话。双方将继续就
2. 经贸务实合作：马克龙此次访华将重点探讨双方合作的未来规划和经贸务实合作，包括贸易、投资、知识产权保护等领
3. 应对全球挑战：马克龙此前表示，应对全球挑战是中法两国共同使命。他此次访华将重点探讨应对气候变化、难民问题
```

那位科技&慕课

大模型问答的挑战及优化策略

外挂文档问答之挑战

大模型之家

➢**版面复杂多样**
  ➢涵盖各类pdf、扫描件、图片等多种复杂文件类型；
  ➢需针对页眉页脚、分栏、列表、跨页等多样化信息进行版面解析；
  ➢需结合OCR能力。

➢**内容复杂、组织多样**
  ➢段落识别；
  ➢列表关系及内容识别。

➢**多因素影响内容召回效果**
  ➢文档内容相似度高(专业文档细分领域、版本迭代等)；
  ➢需要结合文档各级标题信息和段落内容进行匹配；
  ➢文档段落内容较长，影响与query核心信息匹配；
  ➢通用的向量相似度算法效果不好(问题与问题匹配 VS问题与答案匹配)；
  ➢召回率受文档库增大而降低。

➢**有监督样本构造困难**
  ➢需要较强的业务知识；
  ➢内网数据无法利用GPT接口构建。

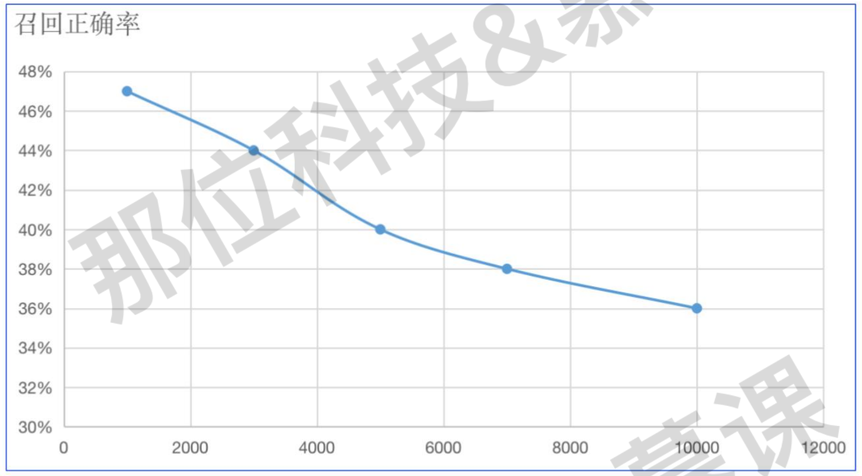
那位科技&慕课

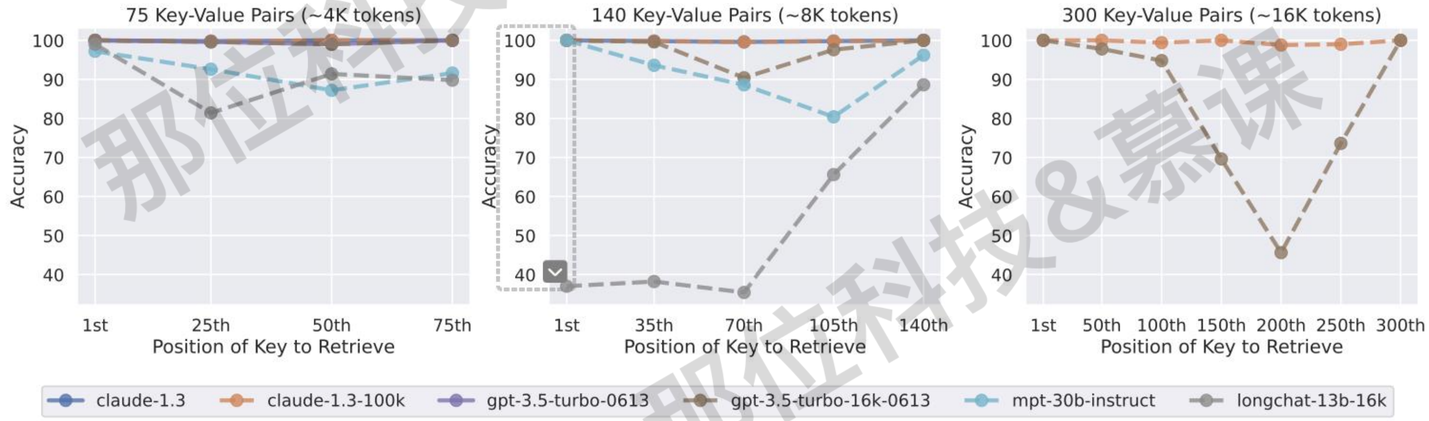
外挂文档问答之挑战

大模型之家

- **召回率受文档数据量增大而降低**
  - 纵坐标为TOP1召回准确率
  - 横坐标为库内文档数量

- **大模型的位置偏见**
  - 相关信息放在开头或者结尾，模型准确性更高，而把相关信息放在中间的话会得到最糟糕的效果。
  - 随着知识文档长度的增强，模型准确性也会相应下降，即便是支持长输入的大模型。





参考： https://arxiv.org/pdf/2307.03172.pdf

那位科技&慕课

外挂文档问答之挑战

大模型之家

➢基于大模型进行智能问答的工作，其核心在于将用户问题和本地知识进行 Embedding，通过向量相似度(Vector Similarity)实现召回，**通过LLM根据召回的内容**回答问题。

➢**特定领域的embedding较难构造**

➢使用通用领域的embedding向量化方法的性能并不高

➢**相似度阈值不易控制**

➢**文档档数很多时，相关性计算上会收到很大挑战**

➢**相似度阈值过高或topk【最终保留的文档数】**过低导致某些有效知识点无法命中，反之不相关的内容与噪声会被引入。

➢**最大输入长度的限制**，通常会截断处理，容易遗漏相似度较低但有效的知识点。

➢为了避免召回结果有遗漏，就需要**降低相似度评分下限**，甚至同时提高召回结果数量上限(top k)，这样会带来明显的问题，包括召回结果有效信息密度大幅降低，并且长度很长，既慢又花钱。

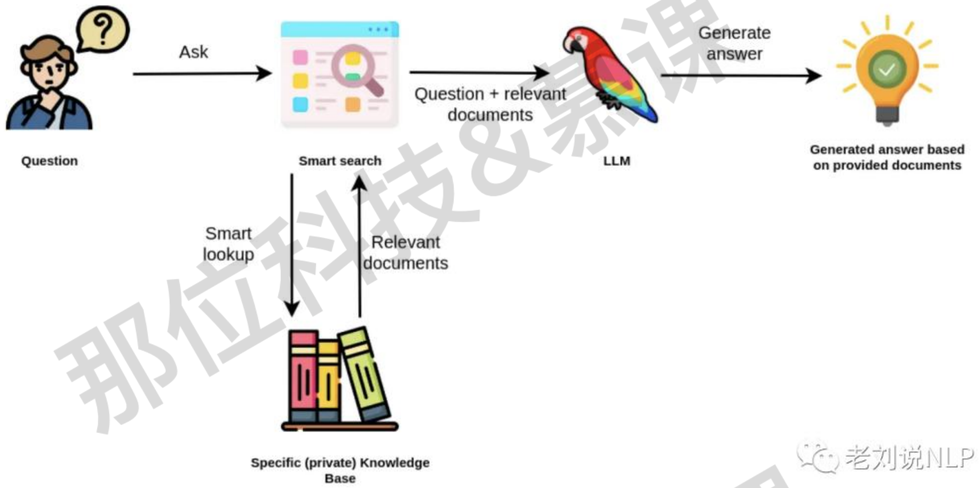
那位科技&慕课

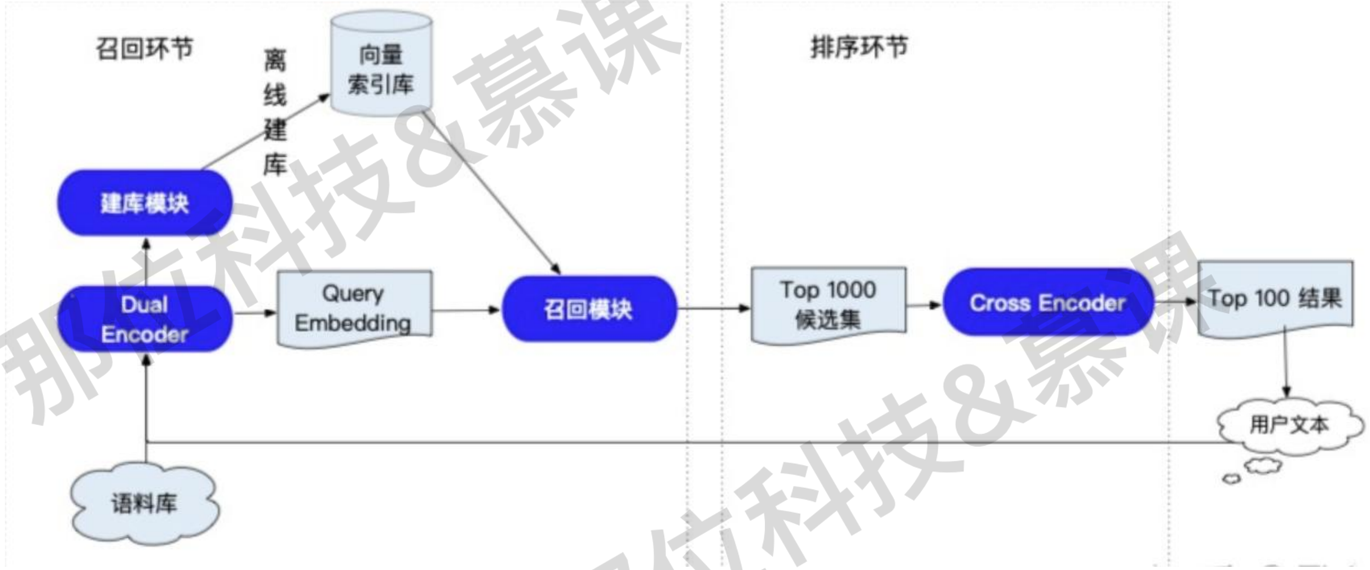
外挂文档问答之挑战

大模型之家

➢“感觉检索阶段已经做好了精确搜索了，按照文中的这种例子，检索出来的答案直接展示就好，感觉不必在过一遍LLM。”

➢“为啥不多路召回，搜索场景非常常用的做法，向量召回效果不一定好，传统方法不能完全丢弃。”





老刘说NLP
那位科技&慕课

外挂文档问答之应对策略

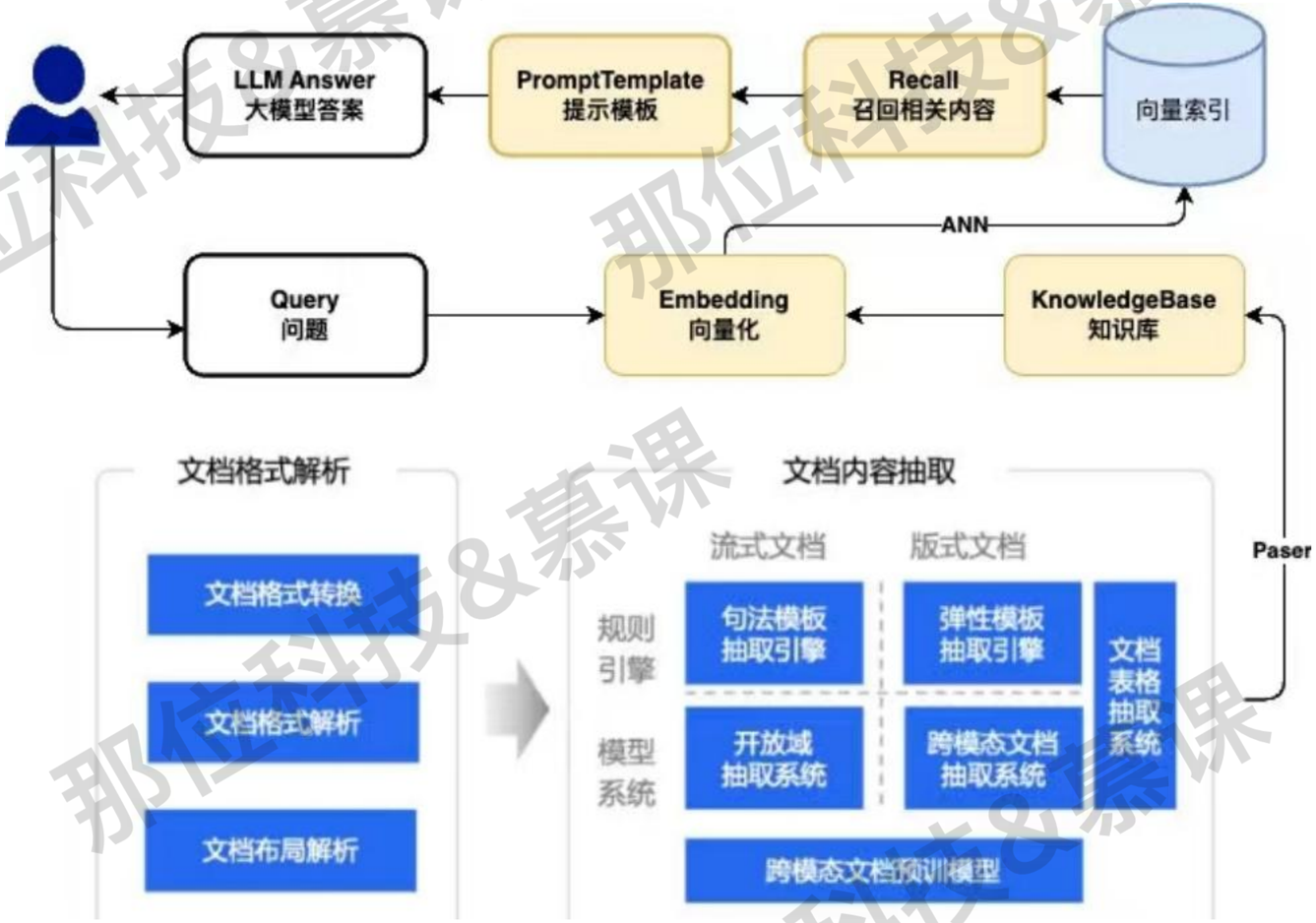
大模型之家

➢预处理侧补丁策略解决文档预处理问题，打标签、关键词问题。

➢向量化侧补丁策略解决领域文档的向量化问题

➢召回侧补丁策略解决文本的召回相关性问题

➢模型侧补丁策略缓解大模型的幻觉解码问题

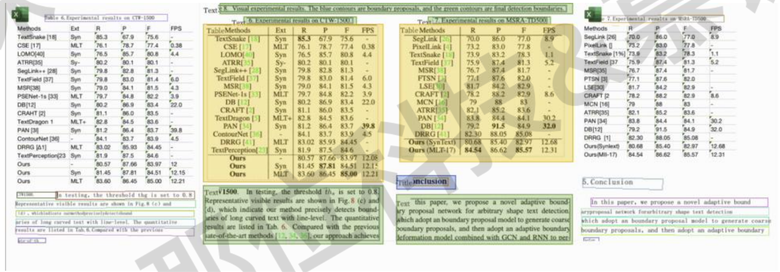


那位科技&慕课

外挂文档问答之应对策略

大模型之家

➢表格解析方法
- ➢pdfplumber
  - ➢地址：https://github.com/jsvine/pdfplumber
- ➢camelot
  - ➢地址：https://github.com/camelot-dev/camelot
- ➢ppstructure
  - ➢地址：https://github.com/PaddlePaddle/PaddleOCR



代号	项目	结果	参考值	单位
ALT	谷丙转氨酶	25.6	0--40	U/L
TBIL	总胆红素	11.2	<20	umol/L
DBIL	直接胆红素	3.3	0--7	umol/L
IBIL	间接胆红素	7.9	1.5--15	umol/L
TP	总蛋白	58.9↓	60--80	g/L
ALB	白蛋白	35.1	33--55	g/L
GLO	球蛋白	23.8	20--30	g/L
A/G	白球比	1.5	1.5--2.5	
ALP	碱性磷酸酶	93	15--112	IU/L
GGT	谷氨酰转肽酶	14.3	<50	U/L
AST	谷草转氨酶	16.3	8--40	U/L
LDH	乳酸脱氢酶	167	114--240	U/L
ADA	腺苷脱氨酶	12.6	4--24	U/L

PDF导出图片 → PP-Structure → 导出excel文件内容

老刘说NLP

代号	项目	结果	参考值	单位
ALT	谷丙转氨酶	25.6	0--40	UA
TBIL	总胆红素	11.2	K20	umol/
DBIL	直接胆红素	3.3	0--7	umol/L
IBIL	间接胆红素	7.9	1.5-15	umol/L
TP	总蛋白	58.94	60--80	8A
ALB	白蛋白	35.1	33-55	81
GLO	球蛋白	23.8	20--30	8
A/G	白球比	1.5	1.5--2.5	
ALP	碱性磷酸	93	15--112	IUA
GGT	谷氨酰转肽酶	14.3	K50	UA
AST	谷草转氨酶	16.3	8--40	A
LDH	乳酸脱氢	167	114-240	UA
ADA	腺苷脱氨酶	12.	--24	U/L

➢表格标准化表示
- ➢markdownt表示
- ➢latex表示
- ➢HTML表示
- ➢text表示
- ➢Mediawiki

```
\begin{table}[]
\begin{tabular}{llllll}
班级 & 姓名 & 性别 & 语文成绩 & 数学成绩 \\
1  & 张三 & 男  & 88   & 88   \\
1  & 李四 & 女  & 78   &
\end{tabular}
\end{table}
```

```
+------+------+------+----------+----------+
| 班级 | 姓名 | 性别 | 语文成绩 | 数学成绩 |
+------+------+------+----------+----------+
| 1    | 张三 | 男   | 88       | 88       |
+------+------+------+----------+----------+
| 1    | 李四 | 女   | 78       |          |
+------+------+------+----------+----------+
```

```
| 班级 | 姓名 | 性别 | 语文成绩 | 数学成绩 |
|------|------|------|----------|----------|
| 1    | 张三 | 男   | 88       | 88       |
| 1    | 李四 | 女   | 78       |          |
```

```
<table class="tg">
<thead>
  <tr>
    <th class="tg-0pky">班级</th>
    <th class="tg-0pky">姓名</th>
    <th class="tg-0pky">性别</th>
    <th class="tg-0pky">语文成绩</th>
    <th class="tg-0pky">数学成绩</th>
  </tr>
</thead>
<tbody>
  <tr>
    <td class="tg-0pky">1</td>
    <td class="tg-0pky">张三</td>
    <td class="tg-0pky">男</td>
    <td class="tg-0pky">88</td>
    <td class="tg-0pky">88</td>
  </tr>
  <tr>
    <td class="tg-0pky">1</td>
    <td class="tg-0pky">李四</td>
    <td class="tg-0pky">女</td>
    <td class="tg-0pky">78</td>
    <td class="tg-0pky"></td>
  </tr>
</tbody>
</table>
```

那位科技&慕课

外挂文档问答之向量化召回

大模型之家

➢https://github.com/shibing624/text2vec/wiki/文本向量表示方法

➢**传统方法：基于特征的匹配**

➢思想：基于TF-IDF、BM25、Jaccord、SimHash、LDA等算法抽取两个文本的词汇、主题等层面的特征，然后使用机器学习模型（LR，xgboost）训练分类模型

➢优点：可解释性较好

➢缺点：依赖人工寻找特征，泛化能力一般，而且由于特征数量的限制，模型的效果比较一般

➢代表模型：BM25

➢通过候选句子的字段对qurey字段的覆盖程度来计算两者间的匹配得分，得分越高的候选项与query的匹配度更好，主要解决词汇层面的相似度问题。

那位科技&慕课

外挂文档问答之向量化召回

大模型之家

➢**深度方法：基于表征的匹配**

➢思想：基于表征的匹配方式，初始阶段对两个文本各自单独处理，通过深层的神经网络进行编码（encode），得到文本的表征（embedding），再对两个表征进行相似度计算的函数得到两个文本的相似度

➢优点：基于BERT的模型通过有监督的Fine-tune在文本表征和文本匹配任务取得了不错的性能

➢缺点：BERT自身导出的句向量（不经过Fine-tune，对所有词向量求平均）质量较低，甚至比不上Glove的结果，因而难以反映出两个句子的语义相似度

➢**BERT对所有的句子都倾向于编码到一个较小的空间区域内**，这使得大多数的句子对都具有较高的相似度分数，即使是那些语义上完全无关的句子对。

➢**BERT句向量表示的聚集现象和句子中的高频词有关**。具体来说，当通过平均词向量的方式计算句向量时，那些高频词的词向量将会主导句向量，使之难以体现其原本的语义。当计算句向量时去除若干高频词时，聚集现象可以在一定程度上得到缓解，但表征能力会下降。

➢代表模型：Sentence-BERT、BERT-flow、SimCSE

那位科技&慕课

外挂文档问答之向量化召回

大模型之家

➢m3e

➢简介：使用in-batch负采样的对比学习的方式在句对数据集进行训练，

➢M3E 在大规模句对数据集上的训练，包含中文百科，金融，医疗，法律，新闻，学术等多个领域共计 2200W 句对样本。

➢地址：https://huggingface.co/moka-ai/m3e-base

	text2vec	openai-ada-002	m3e-small	m3e-base	DMetaSoul	uer	erlangshen
map@1	0.4684	0.6133	0.5574	**0.626**	0.25203	0.08647	0.25394
map@10	0.5877	0.7423	0.6878	**0.7656**	0.33312	0.13008	0.34714
mrr@1	0.5345	0.6931	0.6324	**0.7047**	0.29258	0.10067	0.29447
mrr@10	0.6217	0.7668	0.712	**0.7841**	0.36287	0.14516	0.3751
ndcg@1	0.5207	0.6764	0.6159	**0.6881**	0.28358	0.09748	0.28578
ndcg@10	0.6346	0.7786	0.7262	**0.8004**	0.37468	0.15783	0.39329

那位科技&慕课

外挂文档问答之向量化召回

大模型之家

➢text2vec

➢**简介**：text2vec实现了Word2Vec、RankBM25、BERT、Sentence-BERT、CoSENT等多种文本表征、文本相似度计算模型，并在文本语义匹配（相似度计算）任务上比较了各模型的效果。

➢**Word2Vec**：通过腾讯AILab开源的大规模高质量中文词向量数据（800万中文词轻量版）（文件名：light_Tencent_AILab_ChineseEmbedding.bin密码：tawe）实现词向量检索，对句子采用词向量求平均

➢**SBERT(Sentence-BERT)**：权衡性能和效率的句向量表示模型，训练时通过有监督训练上层分类函数，文本匹配预测时直接句子向量做余弦

➢**CoSENT(CosineSentence)**：CoSENT模型提出了一种排序的损失函数，使训练过程更贴近预测，模型收敛速度和效果比Sentence-BERT更好

➢地址：https://github.com/shibing624/text2vec

Arch	BaseModel	Model	ATEC	BQ	LCQMC	PAWSX	STS-B	SOHU-dd	SOHU-dc	Avg	QPS
Word2Vec	word2vec	w2v-light-tencent-chinese	20.00	31.49	59.46	2.57	55.78	55.04	20.70	35.03	23769
SBERT	xlm-roberta-base	sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2	18.42	38.52	63.96	10.14	78.90	63.01	52.28	46.46	3138
CoSENT	hfl/chinese-macbert-base	shibing624/text2vec-base-chinese	31.93	42.67	70.16	17.21	79.30	70.27	50.42	51.61	3008
CoSENT	hfl/chinese-lert-large	GanymedeNil/text2vec-large-chinese	32.61	44.59	69.30	14.51	79.44	73.01	59.04	53.12	2092
CoSENT	nghuyong/ernie-3.0-base-zh	shibing624/text2vec-base-chinese-sentence	43.37	61.43	73.48	38.90	78.25	70.60	53.08	59.87	3089
CoSENT	nghuyong/ernie-3.0-base-zh	shibing624/text2vec-base-chinese-paraphrase	44.89	63.58	74.24	40.90	78.93	76.70	63.30	**63.08**	3066
CoSENT	sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2	shibing624/text2vec-base-multilingual	32.39	50.33	65.64	32.56	74.45	68.88	51.17	53.67	3138

text2vec的评测效果

那位科技&慕课

外挂文档问答之意图识别

大模型之家

➢**意图识别（多步检索）**

➢不同问题需要不同类型的知识库来辅助问答，首先需要识别出问题意图获取哪些知识。

➢**通过问题中的关键词和知识库特征的关键词匹配，以此识别出问题涉及的知识类型并用对应知识库辅助。**

➢通过识别问题意图，缩小需要检索的知识库范围，减少易混淆知识带来的影响，提升检索精确度。

{"法律法条": ["法律","法条","判罚","罪名","刑期","本院认为","观点","条文","条款","案由","定罪","量刑","法院观点","法官意见","条例","规定","执法依据","法规","罪责","量刑","诉讼","立法"], "法律书籍": ["依据","法律读物","法学著作","法律参考书","法典","法规","参考书","读本","丛书","法理","法学","法哲学","法学家","著作","文献","学说","学术"], "法律文书模板": ["文书","起诉书","法律文书","起诉状","判决书","裁定书","答辩状","法律合同","协议","证据","证明","合同","格式","模板","样本","规范","范本"], "法律案例": ["法律","判罚","事实","案例","罪名","刑期","本院认为","观点","法律案件","典型案例","案情","案由","定罪","量刑","证据","判例","裁决","仲裁","先例","判决","司法"], "法律考试": ["选项","选择","A,B,C,D","ABCD","A,B,C和D","考试","题目","法考","法律考试","考题","选择题","判断题","多选题","单选题","填空题","辨析题","案例分析题","答案","试题","试卷","法学","考研","司法考试","律师考试"]}

外挂文档问答之意图识别

大模型之家

➢**基于大模型prompt或有监督模型进行意图分类）**

➢采用统计特征层面的检索和语义特征层面的检索。

➢对于统计特征，预先提取知识库中每条知识的关键词，比如法条库中每条法条的关键词是所属法律和法条条数，使用fuzzy matching提取问题中的关键词和知识关键词匹配获取相关知识；

➢对于语义特征，使用向量相似度检索，为了提升检索精确度，**预先准备每条知识对应的摘要，向量检索时使用知识摘要和问题进行相似度计算，找到相关知识后替换成具体知识。**

请问“{}”是属于下面哪个类别的问题?
A: 基本信息查询, 例如证券信息、股票简称、股票代码、外文名称、法定代表人、注册地址、办公地址、公司网址、电子信箱等
B: 公司员工人数统计, 例如员工人数、员工专业、员工教育程度等
C: 财务相关, 例如金额、费用、资产、收入等
D: 以上都不是
例如:
1. XXXX的费用收入是多少元?
输出: C
2. XX公司法定代表人是谁?
输出: A
3. 请简要介绍分析XX公司的XXX情况。
输出: D
4. XX公司硕士人数是什么?
输出: B
你只需要回答编号, 不要回答其他内容.

那位科技&慕课

外挂文档问答之生成FAQ

大模型之家

➢结构化数据生成qa对
  ➢人工定义模板，快速生成QA对；
  ➢让模型针对q进行改写

Instruction Generation Prompt

The background knowledge is:
{unsupervised knowledge data}

Please generate ten instruction questions as diverse as possible based on the content of the above article. These questions can be questions about facts or an understanding and evaluation of relevant content. Please assume that there is no corresponding article to refer to when asking questions, so do not use demonstrative pronouns such as “this” or “these” in the question.

Please generate questions in the following format:
1. Question: ...
2. Question: ...

➢基于大模型prompt做self-instruct生成加人工核对
  ➢给定文本，让大模型生成问题
  ➢根据问题，让大模型生成答案
  ➢人工检验核对

Reading Comprehension Prompt

The background knowledge is:
{unsupervised knowledge data}
Please answer the following question based on the content of the article above:
{the generated question}

Please answer this question as thoroughly as possible, but do not change the key information in the original text, and do not include expressions such as “based on the above article” in the answer.

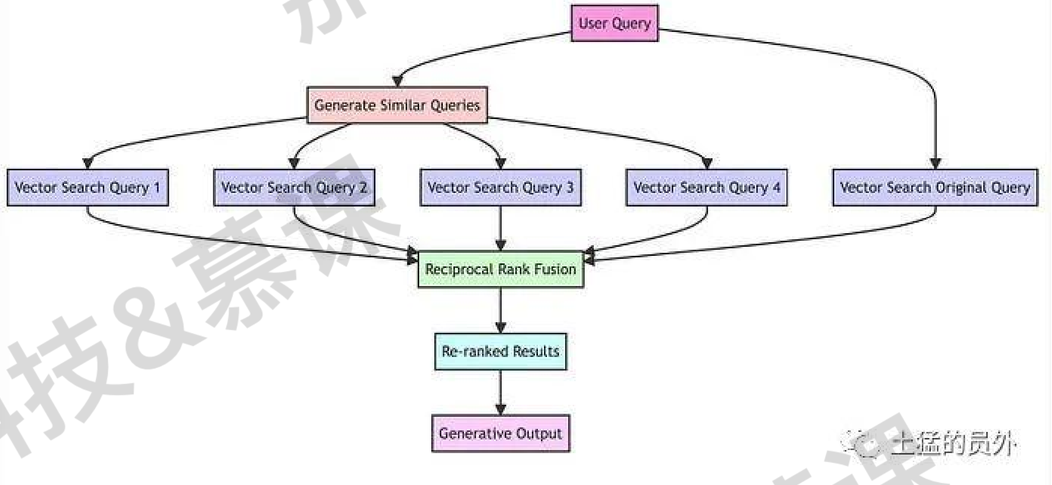
Please generate the corresponding answer in the following format:
Question: ...
Answer: ...

那位科技&慕课

外挂文档问答之多路召回

大模型之家

- **方案**
  - 查询语句扩展
    - 通过LLM将用户的查询转换为相似但不同的查询。
  - 并发的向量搜索
    - 对原始查询及其新生成的同级查询执行并发的向量搜索。
  - 重新排序
    - 聚合和细化所有结果使用倒数排序融合(RRF)。
    - **RRF支持对多种不同方式召回的多个结果集进行综合再排序，返回最终的排序**
  - 最后优中选优
    - 将排序的结果与新查询配对，引导LLM进行有针对性的查询语句输出，考虑所有查询和重新排序的结果列表。



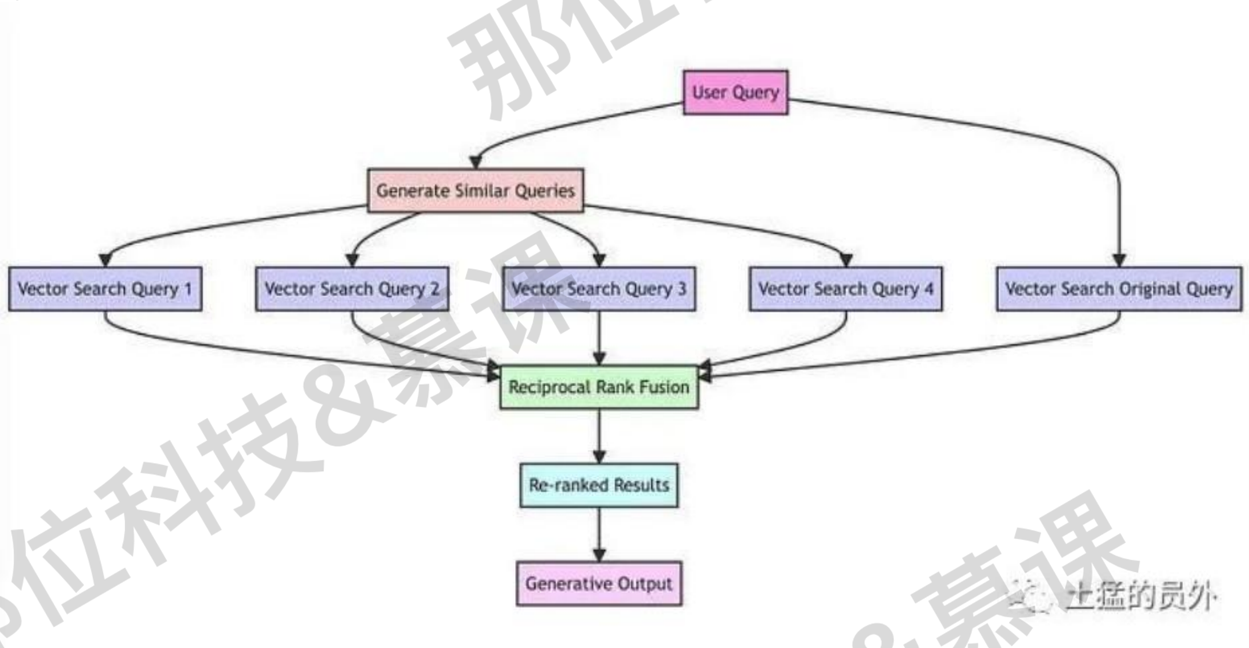
那位科技&慕课

外挂文档问答之多路召回

大模型之家

➢**优点**
- ➢增强用户意图对齐
- ➢自动为用户查询输入纠错
- ➢导航复杂查询（自动分解长句的意图）
- ➢搜索中的意外发现（关联推荐）

➢**缺点**
- ➢过于啰嗦、信息泛滥
- ➢成本会昂贵，多查询输入tokens消耗



那位科技&慕课

外挂文档问答之多路召回

大模型之家

➢RRF支持对多种不同方式召回的多个结果集进行综合再排序，返回最终的排序

➢k是一个常数值，默认是60，R包含每个查询结果的doc排序集合。r(d)表示doc d在某个查询条件中排序号，排序次序从1开始。

$$RRFscore(d \in D) = \sum_{r \in R} \frac{1}{k + r(d)}$$

D - set of docs
R - set of rankings as permutation on 1..|D|
K - typically set to 60 by default

BM25 相关性排名	密集向量相关性排名	RRF结果 k=0
A 1	B 1	B：1/2 + 1/1 = 1.5
B 2	C 2	A：1/1 + 1/3 = 1.3
C 3	A 3	C：1/3 + 1/2 = 0.83

那位科技&慕课

外挂文档问答之多路召回

大模型之家

➢RRF支持对多种不同方式召回的多个结果集进行综合再排序，返回最终的排序
➢对于查询"hydrogen is a liquid below what temperature"，三种查询方式返回结果和准确性。

```
// RRF混排查询
GET collection-with-embeddings/_search
{
  "size": 10,
  "query": {
    "query_string": {
      "query": "hydrogen is a liquid below what temperature"
    }
  },
  "knn": [
    {
      "field": "text_embedding.predicted_value",
      "k": 10,
      "num_candidates": 100,
      "query_vector_builder": {
        "text_embedding": {
          "model_id": "sentence-transformers__msmarco-minilm-l-12-v3",
          "model_text": "hydrogen is a liquid below what temperature"
        }
      }
    }
  ],
  "_source": [
    "id"
  ],
  "rank": {
    "rrf": {
      "window_size": 10,
      "rank_constant": 1
    }
  }
}
```

```
//向量查询
GET collection-with-embeddings/_search
{
  "size": 10,
  "knn": [
    {
      "field": "text_embedding.predicted_value",
      "k": 10,
      "num_candidates": 100,
      "query_vector_builder": {
        "text_embedding": {
          "model_id": "sentence-transformers__msmarco-minilm-l-12-v3",
          "model_text": "hydrogen is a liquid below what temperature"
        }
      }
    }
  ],
  "_source": [
    "id"
  ]
}
//文本查询
GET collection-with-embeddings/_search
{
  "size": 10,
  "query": {
    "query_string": {
      "query": "how are some sharks warm blooded"
    }
  },
  "_source": [
    "id"
  ]
}
```

那位科技&慕课

外挂文档问答之多路召回

大模型之家

➢准确性分为四个等级：不相关（0）、相关（段落切题但没有回答问题）（1）、高度相关（2）和完全相关（3）。准确性分数越高越相关。

➢RRF的排序结果，在向量查询结果的基础上，结合了文本查询的结果，将没有在向量召回出现的结果"7911557" 排到了前面，提高了召回的准确性。

➢RRF的排序结果，**将没有在文本查询中出现的结果"6080460" 排到了前面，提高了召回的准确性。**

RRF混排		向量召回		文本查询	
段落ID	准确性	段落ID	准确性	段落ID	准确性
8588222	0	8588222	0	**7911557**	**3**
8588219	3	8588219	3	8588219	3
7911557	**3**	**6080460**	**3**	8588222	0
128984	3	128984	3	2697752	2
6080460	**3**	4254815	1	128984	3
2697752	2	6343521	1	1721142	0
4254815	1	1020793	0	8588227	0
1721142	0	4254811	3	302210	1
6343521	1	1959030	0	2697746	2
8588227	0	4254813	1	7350325	0

那位科技&慕课

外挂文档问答之引入拒答

大模型之家

➢如果LLM在预训练阶段没有获得相关的先决知识，在训练指令时是一个错误的对齐过程，会促使LLM产生幻觉

➢SFT可用框架：https://github.com/PaddlePaddle/PaddleNLP/tree/develop/llm

➢**加工整理数据**

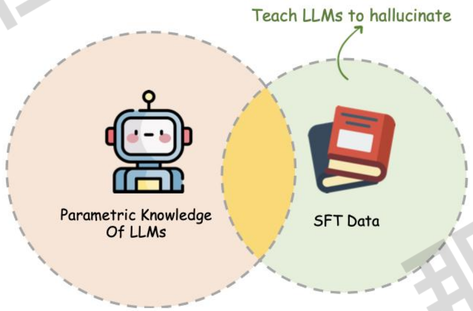
➢SFT数据量相对较小，手动和自动整理都是可行的选择。经过编辑的指令数据上进行微调的LLM表现出更高的真实性和事实性水平。

➢**加入拒答**

➢在推理过程中，如果被要求回答与未学知识相关的问题，大模型很可能会自信地产生幻觉。

➢采用以诚实为导向的SFT，即在SFT数据中引入一些诚实样本。诚实样本指的是承认自己无能的回答，如"对不起，我不知道"。

➢学会拒绝回答特定问题减少幻觉。



参考:《Siren's Song in the AI Ocean: A Survey on Hallucination in Large Language Models》，https://arxiv.org/abs/2309.01219

那位科技&慕课

外挂文档问答之引入拒答

大模型之家

➢如果LLM在预训练阶段没有获得相关的先决知识，在训练指令时是一个错误的对齐过程，会促使LLM产生幻觉

➢SFT可用框架：https://github.com/PaddlePaddle/PaddleNLP/tree/develop/llm

```
# 鼓励回答数据示例
{
    "prompt": "回答下列问题。注意：你只有在你非常确定答案的时候才回答，否则请回复你不知道。\n问题：岳云的父亲是？",
    "label": "岳云的父亲是岳飞，岳飞是南宋时期的一位著名将领和抗金英雄。\n岳飞出生于北宋末年，..."
}

# 鼓励拒答数据示例
{
    "prompt": "回答下列问题。注意：你只有在你非常确定答案的时候才回答，否则请回复你不知道。\n问题：阿道夫·奥瓦列的出生地点在哪里？"
    "label": "我不知道和“阿道夫·奥瓦列的出生地点”相关的信息。"
}
```

那位科技&慕课

那位科技

thanks 👋